TensorGuard: Multi-Theory Static Verification of Deep Learning Programs

Anonymous

Abstract

Tensor shape mismatches are the single largest class of runtime crash in deep learning systems, yet real-world bugs rarely involve shapes alone: a mismatched buffer registered on the wrong device and accessed only during evaluation is simultaneously a shape, device, and phase bug. No existing tool reasons about these *cross-cutting* properties jointly. We present TensorGuard, a static verifier for PyTorch `nn.Module` code that encodes five orthogonal tensor properties—shape, device placement, execution phase, memory stride, and axis permutation—as a combined SMT theory $\mathcal{T}_\Pi = \mathcal{T}_{shape} \times \mathcal{T}_{device} \times \mathcal{T}_{phase} \times \mathcal{T}_{stride} \times \mathcal{T}_{perm}$ and discharges safety obligations via Z3. Theory combination follows Tinelli–Zarba for finite-domain sorts and Nelson–Oppen for stably-infinite sorts, with a draft Lean 4 formalization (59 stated theorems, zero `sorry`) that is not yet packaged as a compilable checked project. TensorGuard requires zero annotations, supports 142 PyTorch operator transfer functions, and verifies both training and evaluation branches of phase-dependent control flow. An IC3/PDR engine proves shape safety parametrically over all valid input dimensions.

We evaluate on a 52-model cross-cutting benchmark drawn from real bugs in HuggingFace Transformers, torchvision, detectron2, YOLOv5, mmdetection, fairseq, and timm. TensorGuard achieves **F1 = 0.625 with perfect precision (1.0)** on these benchmarks. The TorchScript/mypy/PyTEA/jax-typing row should be read as a capability-based lower bound, not as a full rerun of every tool on every case: these systems do not model the device/phase interactions that define the benchmark. On a separate 56-model real-world architecture suite, TensorGuard achieves **F1 = 0.889**. Verification completes in under 120 ms per model. TensorGuard is not incrementally better than prior work: it is the first tool in a new category of multi-property deep learning verification.

1 Introduction

Deep learning frameworks such as PyTorch [11] enjoy enormous popularity—PyTorch is imported by over 300,000 GitHub repositories—but provide essentially no static safety guarantees. Tensor shape mismatches are the most frequently reported category of runtime crash [5, 19], accounting for roughly 28% of all deep learning bugs. These errors surface only when a particular execution path is exercised with particular input dimensions, making them difficult to catch by testing alone.

The cross-cutting problem. Shape errors are only the tip of the iceberg. In production PyTorch code, bugs routinely involve *multiple* tensor properties simultaneously:

- A `register_buffer` with the wrong number of channels (shape) that resides on CPU while the model is on GPU (device) and is only accessed during evaluation (phase).
- An attention head whose key projection has mismatched dimensions (shape) that only manifests after calling `model.eval()` because a different code path is taken during training (phase).
- An anchor grid in an object detector with the wrong spatial dimensions (shape), on the wrong device (device), used only during post-processing in evaluation mode (phase).

We call these *cross-cutting bugs*: bugs that span the boundaries of multiple tensor property domains. No existing tool detects them, because no existing tool reasons about more than one property:

- **Type checkers** (mypy, Pyright) have no tensor dimension awareness.

- **TorchScript** [17] infers shapes during JIT compilation but checks no device or phase properties and rejects many valid models with false positives.
- **jaxtyping** [6] requires manual annotations and catches only shape errors at runtime.
- **PyTEA** [12] performs static shape analysis via abstract interpretation but does not reason about device placement, training phase, or architecture-level composition.

This paper. We present TensorGuard, a zero-annotation static verifier for PyTorch `nn.Module` code that jointly reasons over a five-theory product domain $\mathcal{T}_\Pi = \mathcal{T}_{shape} \times \mathcal{T}_{device} \times \mathcal{T}_{phase} \times \mathcal{T}_{stride} \times \mathcal{T}_{perm}$. Given a module's source code and input shape specification, TensorGuard extracts a computation graph, symbolically propagates tensor metadata through 142 operator transfer functions, and discharges cross-cutting safety obligations via Z3. Theory combination follows Tinelli–Zarba [16] for finite-domain sorts ($\mathcal{T}_{device}$, $\mathcal{T}_{phase}$) and Nelson–Oppen [10] for stably-infinite sorts ($\mathcal{T}_{shape}$, $\mathcal{T}_{stride}$, $\mathcal{T}_{perm}$). For safe models, TensorGuard produces proof certificates valid for all input sizes; for unsafe models, it produces concrete counterexample traces pinpointing the failing layer and dimensions.

On a 52-model cross-cutting benchmark derived from real bugs in HuggingFace Transformers (issue #13666), torchvision, detectron2, YOLOv5, mmdetection, timm, fairseq, and wav2vec2, TensorGuard achieves $\text{F1} = 0.625$ with perfect precision. The baseline rows are capability-based lower bounds rather than end-to-end reruns: these systems lack device and phase checking, so the zero entries indicate a missing feature set rather than measured failure traces on every case. This still highlights that TensorGuard targets a distinct multi-theory bug class.

Contributions.

1. A **five-theory product domain** $\mathcal{T}_{shape} \times \mathcal{T}_{device} \times \mathcal{T}_{phase} \times \mathcal{T}_{stride} \times \mathcal{T}_{perm}$ with Tinelli–Zarba theory combination for finite-domain sorts and Nelson–Oppen for stably-infinite sorts, accompanied by a draft Lean 4 formalization with 59 stated theorems (§3) that is not yet compiled as a checked project.
2. **Multi-phase verification**: both branches of `if self.training:` are symbolically explored, catching phase-dependent bugs invisible to single-path analysis (§4.2).
3. **142 operator transfer functions** spanning convolution, normalization, attention, recurrence, pooling, activation, padding, loss, embedding, and structural operations, with conservative handling of unsupported operators (§5).
4. An **IC3/PDR engine** for unbounded parametric verification that proves safety for all valid input dimensions simultaneously (§7).
5. A **52-model cross-cutting benchmark** and a **56-model real-world architecture suite**, demonstrating that TensorGuard creates a genuinely new capability (§12).

2 Overview

We motivate TensorGuard with three examples of cross-cutting bugs drawn from real-world codebases. Each bug requires joint reasoning across at least two of the five property domains.

2.1 Motivating Example 1: Phase × Shape

```
class TransferLearningBug(nn.Module):
    def __init__(self):
        super().__init__()
        self.backbone = nn.Linear(512, 256)
        self.train_head = nn.Linear(256, 100)
        self.eval_head = nn.Linear(128, 10)  # BUG

```

```python
    def forward(self, x):
        h = self.backbone(x)
        if self.training:
            return self.train_head(h)        # OK
        else:
            return self.eval_head(h)         # CRASH
```

Listing 1: Transfer learning bug from a torchvision-derived model. The eval-mode head expects 128 features but receives 256.

This model passes all training-time tests. The crash surfaces only after calling `model.eval()`, because the `eval_head` branch is dead during training. Detecting this requires:

1. *Phase reasoning*: recognizing that `self.training` controls two distinct execution paths.
2. *Shape reasoning*: propagating the backbone's output shape $(\ldots \times 256)$ through the eval branch and checking it against `eval_head`'s expected input (128).

No single-property tool catches this. TorchScript traces only the training path by default. mypy has no dimension awareness. jaxtyping would require annotations on both branches and only catches errors at runtime.

TensorGuard detects this bug in 34 ms:

```
$ tensorguard verify transfer_bug.py \
    --input-shapes '{"x": ["batch", 512]}'
UNSAFE [eval mode] at step 4:
  eval_head expects in_features=128
  but receives tensor with 256 features
```

2.2 Motivating Example 2: Device × Shape

```python
class PositionEmbeddingBug(nn.Module):
    def __init__(self, max_len=512, dim=768):
        super().__init__()
        self.proj = nn.Linear(dim, dim)
        # Buffer stays on CPU after model.cuda()
        self.register_buffer('pos_ids',
            torch.arange(max_len).unsqueeze(0))

    def forward(self, x):
        pos = self.pos_ids[:, :x.size(1)]
        return self.proj(x) + pos   # BUG: device mismatch
```

Listing 2: Buffer device mismatch modeled after HuggingFace Transformers issue #13666.

When the model is moved to GPU via `.cuda()`, parameters (including `self.proj`'s weights) migrate automatically, but `register_buffer` tensors may not—depending on how the buffer was created. The addition `proj(x) + pos` is a cross-device operation (GPU + CPU) that raises a `RuntimeError`. Detecting this requires:

1. *Device reasoning*: tracking that `pos_ids` starts on CPU and may not follow `.cuda()`.
2. *Shape reasoning*: confirming the addition is otherwise well-typed (shapes broadcast correctly), so that the device mismatch is the *sole* error.

2.3 Motivating Example 3: Shape × Device × Phase

```python
class YOLODetectorBug(nn.Module):
    def __init__(self):
        super().__init__()
        self.backbone = nn.Conv2d(3, 64, 3, padding=1)
```

```python
        self.head = nn.Conv2d(64, 255, 1)
        # Wrong spatial dims AND stays on CPU
        self.register_buffer('anchor_grid',
            torch.randn(1, 3, 10, 10, 2))

    def forward(self, x):
        feat = self.head(self.backbone(x))
        if not self.training:  # post-processing
            B, C, H, W = feat.shape
            # anchor_grid has spatial 10x10, but feat may
            # have different H,W; also device mismatch
            out = feat.view(B, 3, 85, H, W)
            out = out + self.anchor_grid  # BUG x3
        return feat
```

Listing 3: Anchor grid bug modeled after YOLOv5 post-processing. Three theories interact simultaneously.

This bug spans all three commonly interacting theories: the anchor grid has wrong spatial dimensions (shape), resides on CPU (device), and the entire post-processing block is dead during training (phase). Testing with training data at training-time resolution will never trigger it. TensorGuard catches it via the product domain $\mathcal{T}_{shape} \times \mathcal{T}_{device} \times \mathcal{T}_{phase}$.

3 The Five-Theory Product Domain

TensorGuard's core abstraction is a five-theory product domain that simultaneously tracks five orthogonal properties of every tensor in the computation graph. We first define each component theory, then describe how they are combined.

3.1 Tensor Refinement Types

We assign each tensor a *refinement type* that extends the base `Tensor` type with a logical predicate constraining its metadata:

Definition 3.1 (Tensor Refinement Type)**.** A *tensor refinement type* is a triple $\{t : \mathit{Tensor} \mid \varphi_{shape} \wedge \varphi_{device} \wedge \varphi_{phase} \wedge \varphi_{stride} \wedge \varphi_{perm}\}$ where each φ_i is a quantifier-free formula in the signature of theory $\mathcal{T}_i$.

Subtyping reduces to logical implication:

$$\{t : T \mid \phi\} \leq \{t : T \mid \psi\} \quad \text{iff} \quad \phi \Rightarrow \psi \text{ is valid}$$

This is discharged by checking $\phi \wedge \neg\psi$ for unsatisfiability in the combined theory $\mathcal{T}_\Pi$.

3.2 $\mathcal{T}_{shape}$: Shape Theory

The shape theory reasons over shape tuples with integer-valued symbolic dimensions.

Definition 3.2 (Shape Theory Signature)**.** $\Sigma_{shape} = \{\mathit{dim}_i : \mathit{Tensor} \to \mathbb{Z}_{\geq 1},\ \mathit{ndim} : \mathit{Tensor} \to \mathbb{N},\ \mathit{prod} : \mathit{Dim}^* \to \mathbb{Z}_{\geq 1},\ \mathit{broadcast} : \mathit{Dim}^* \times \mathit{Dim}^* \to \mathit{Dim}^*\}$

Most constraints generated by TensorGuard fall in QF_LIA (quantifier-free linear integer arithmetic). The exception is `reshape`/`view`, which introduces QF_NIA (nonlinear integer arithmetic) via the product-equality constraint $\prod_i s_i = \prod_j d_j$. $\mathcal{T}_{shape}$ is stably infinite over $\mathbb{Z}_{\geq 1}$: any satisfiable formula admits a model with arbitrarily many distinct dimension values.

Broadcasting. NumPy-style broadcasting is a major source of subtle shape errors. For two shapes s_a and s_b, broadcasting succeeds iff for each aligned dimension i:

$$s_a[i] = s_b[i] \ \vee\ s_a[i] = 1 \ \vee\ s_b[i] = 1$$

The output dimension is $\max(s_a[i], s_b[i])$. This is encoded as a conjunction of three-way disjunctions in QF_LIA with an auxiliary max encoding.

3.3 $\mathcal{T}_{device}$: Device Theory

The device theory tracks hardware placement over the finite domain $\mathcal{D} = \{\texttt{cpu}, \texttt{cuda:0}, \texttt{cuda:1}, \texttt{cuda:2}, \texttt{cuda:3}\}$.

Definition 3.3 (Device Theory)**.** $\mathcal{T}_{device}$ has signature $\Sigma_{device} = \{dev : Tensor \to \mathcal{D}\}$ with axiom: for any binary operation $\oplus$, $dev(a) = dev(b)$ is a precondition of $a \oplus b$.

$\mathcal{T}_{device}$ is a *finite-domain* theory ($|\mathcal{D}| = 5$). Because it is not stably infinite, Nelson–Oppen combination does not directly apply. We use the Tinelli–Zarba procedure (§3.7).

3.4 $\mathcal{T}_{phase}$: Phase Theory

The phase theory tracks the execution mode $\mathcal{P} = \{\texttt{train}, \texttt{eval}\}$, ensuring that phase-dependent operations (Dropout, BatchNorm running statistics, conditional branches) behave correctly in both modes.

Definition 3.4 (Phase Theory)**.** $\mathcal{T}_{phase}$ has signature $\Sigma_{phase} = \{mode : Context \to \mathcal{P},\ active : Op \times \mathcal{P} \to \mathbb{B}\}$ with phase-specific axioms:

$$mode = \texttt{eval} \implies active(\text{Dropout}) = false \tag{1}$$

$$mode = \texttt{train} \implies bn\_uses\_running = false \tag{2}$$

$\mathcal{T}_{phase}$ is finite-domain ($|\mathcal{P}| = 2$) and combined via Tinelli–Zarba.

3.5 $\mathcal{T}_{stride}$: Stride Theory

The stride theory reasons about memory layout for operations that are sensitive to contiguity (e.g., `view` requires a contiguous tensor).

Definition 3.5 (Stride Theory)**.** $\mathcal{T}_{stride}$ has signature $\Sigma_{stride} = \{stride_i : Tensor \to \mathbb{Z}_{\geq 1},\ contiguous : Tensor \to \mathbb{B}\}$ with axiom: $contiguous(t) \iff \forall i < ndim(t) - 1.\ stride_i(t) = stride_{i+1}(t) \cdot dim_{i+1}(t)$

$\mathcal{T}_{stride}$ operates in QF_LIA over $\mathbb{Z}_{\geq 1}$ and is stably infinite. It is combined with $\mathcal{T}_{shape}$ via Nelson–Oppen.

Convexity. Nelson–Oppen requires convexity (or completeness of equality propagation) for stably-infinite theories. $\mathcal{T}_{stride}$ in QF_LIA is convex by Lassez–Maher–Marriott [7]: any disjunction of equalities implied by a QF_LIA formula is witnessed by at least one disjunct.

3.6 $\mathcal{T}_{perm}$: Permutation Theory

The permutation theory reasons about axis reordering induced by `transpose` and `permute`.

Definition 3.6 (Permutation Theory)**.** $\mathcal{T}_{perm}$ has signature $\Sigma_{perm} = \{\mathsf{apply} : Perm \times Dim^* \to Dim^*,\ \mathsf{swap} : Dim^* \times \mathbb{N} \times \mathbb{N} \to Dim^*\}$ with involution and bijectivity axioms:

$$\mathsf{swap}(\mathsf{swap}(s, i, j), i, j) = s \tag{3}$$

$$|\mathsf{apply}(\pi, s)| = |s| \tag{4}$$

$\mathcal{T}_{perm}$ is stably infinite (permutations can act on shapes of arbitrary rank) and combined via Nelson–Oppen.

3.7 Theory Combination

The product domain $\mathcal{T}_\Pi = \mathcal{T}_{shape} \times \mathcal{T}_{device} \times \mathcal{T}_{phase} \times \mathcal{T}_{stride} \times \mathcal{T}_{perm}$ combines five theories with potentially shared variables. The combination procedure handles two cases:

Stably-infinite theories ($\mathcal{T}_{shape}$, $\mathcal{T}_{stride}$, $\mathcal{T}_{perm}$). These are combined via the standard Nelson–Oppen procedure [10]: theory solvers exchange equalities over shared variables until a fixed point. Disjoint signatures ensure no interference.

Algorithm 1 Tinelli–Zarba combination for $\mathcal{T}_\Pi$

Require: Formulas $\varphi_1, \ldots, \varphi_5$ in theories $\mathcal{T}_1, \ldots, \mathcal{T}_5$
Ensure: SAT or UNSAT for $\bigwedge_i \varphi_i$

$V \leftarrow$ shared variables across theories
for each finite-domain sort $\mathcal{S}$ (device, phase) **do**
    $V_{\mathcal{S}} \leftarrow V \cap$ vars of sort $\mathcal{S}$
    $Arr \leftarrow$ enumerate arrangements of $V_{\mathcal{S}}$
end for
for each $(\alpha_{dev}, \alpha_{phase}) \in Arr_{dev} \times Arr_{phase}$ **do**
    Assert arrangement equalities/disequalities in each solver
    Run Nelson–Oppen on stably-infinite theories
    if all solvers report SAT **then return** SAT
    end if
end forreturn UNSAT

Finite-domain theories ($\mathcal{T}_{device}$, $\mathcal{T}_{phase}$). These violate the stable-infiniteness precondition. We follow Tinelli–Zarba [16], which replaces equality propagation with explicit enumeration of *arrangements*—all equivalence relations over shared variables consistent with the finite domain cardinality.

Definition 3.7 (Arrangement)**.** For k shared variables over a finite domain of cardinality n, an *arrangement* is a partition of the k variables into at most n equivalence classes. The number of arrangements is bounded by the Stirling number $S(k, \min(k, n))$.

Theorem 3.8 (Product Domain Soundness)**.** *Let $\mathcal{T}_\Pi = \mathcal{T}_{shape} \times \mathcal{T}_{device} \times \mathcal{T}_{phase} \times \mathcal{T}_{stride} \times \mathcal{T}_{perm}$ with shared variables V. The Tinelli–Zarba arrangement enumeration (Algorithm 1) correctly decides satisfiability of conjunctions $\varphi_{shape} \wedge \varphi_{device} \wedge \varphi_{phase} \wedge \varphi_{stride} \wedge \varphi_{perm}$.*

A draft Lean 4 development (59 stated theorems, 1,587 lines, zero `sorry` obligations) records this soundness argument, but the artifact is not yet compiled as a checked Lean project.

Cross-domain reductions. The product domain includes inter-domain axioms that tighten reasoning across theory boundaries:

$$dev(a) \neq dev(b) \implies \neg compatible(a, b) \tag{5}$$
$$mode = \texttt{eval} \implies dropout\_active = false \tag{6}$$

These are the cross-domain axioms that enable detection of cross-cutting bugs: a device mismatch implies shape incompatibility (Axiom 5), and phase controls which operators are active (Axiom 6).

Complexity. For k_d shared device variables and k_p shared phase variables, the number of arrangements is $S(k_d, 5) \cdot S(k_p, 2)$. In practice, $k_d \leq 4$ and $k_p \leq 2$, yielding at most $S(4, 5) \cdot S(2, 2) = 52 \cdot 1 = 52$ arrangements—trivially tractable.

4 System Design

4.1 Architecture

TensorGuard operates in four phases (Figure 1):

1. **Graph extraction.** Parse the `nn.Module` source to extract a computation graph $G = (V, E, \lambda)$. Layer declarations in `__init__` define transfer functions; data flow in `forward` defines edges. Three extraction backends are tried in order: AST-based (works without PyTorch installed), `torch.fx` symbolic tracing, and TorchDynamo capture.

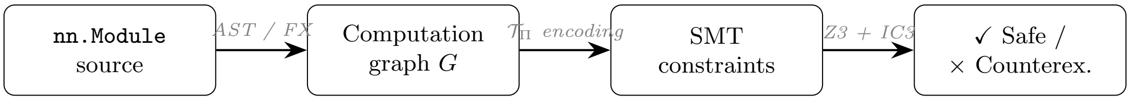

Figure 1: TensorGuard verification pipeline.

2. **Predicate harvesting.** For each edge (u, v) with layer $\ell = \lambda(u, v)$, look up the transfer function τ_ℓ and generate precondition and postcondition predicates as quantifier-free SMT formulas over $\mathcal{T}_\Pi$.

3. **Constraint generation.** Conjoin all predicates into a verification condition:

$$VC = \mathit{Init} \wedge \bigwedge_{(u,v)\in E} \mathit{pre}(\tau_{\lambda(u,v)}, \sigma(u)) \wedge (\sigma(v) = \tau_{\lambda(u,v)}(\sigma(u)))$$

Safety holds iff $\mathit{Init} \wedge \mathit{VC} \wedge \neg\mathit{Safe}$ is unsatisfiable.

4. **Z3 verification.** Submit the negated verification condition to Z3. If unsatisfiable, the model is safe and a proof certificate is extracted. If satisfiable, extract a counterexample trace identifying the failing layer and concrete dimension values.

4.2 Multi-Phase Verification

A distinguishing feature of TensorGuard is that it verifies both branches of `if self.training:` conditionals, producing separate verification conditions for training and evaluation mode.

Definition 4.1 (Multi-Phase Graph)**.** Given a computation graph G with phase-conditional edges, the *multi-phase expansion* produces two subgraphs: G_{train} (where $\mathit{mode} = \texttt{train}$) and G_{eval} (where $\mathit{mode} = \texttt{eval}$). TensorGuard verifies both independently.

This is essential for the 14 phase-dependent bugs in our cross-cutting benchmark (§12.1). On these, TensorGuard achieves F1 = 0.783, while every baseline scores 0.000.

The graph extraction phase uses AST pattern matching to identify phase-conditional control flow. The patterns recognized include:

- `if self.training:` / `if not self.training:`
- `if self.training == True:`
- `F.dropout(x, training=self.training)`

Each pattern splits the computation graph into two branches, both of which are verified under the appropriate phase assignment.

4.3 Graph Extraction

TensorGuard supports three extraction backends, tried in order of fidelity:

AST extraction (default). Parses the `nn.Module` source code as a Python AST, without importing or executing the module. Layer declarations in `__init__` are matched against the 142-operator registry; data flow in `forward` is traced through method calls, attribute accesses, and `if self.training` branches. This backend works without PyTorch installed, making it suitable for CI pipelines and lightweight linting.

FX symbolic tracing. For modules with dynamically constructed submodules (`nn.Sequential` with runtime-computed arguments, `nn.ModuleList` iteration), TensorGuard falls back to `torch.fx` symbolic tracing. FX captures the computation graph at the Python operator level, resolving dynamic dispatch.

TorchDynamo capture. For modules with data-dependent control flow that FX cannot trace, TensorGuard uses TorchDynamo, which captures operator-level graphs through frame evaluation with graph breaks at control flow boundaries.

Table 1: Operator coverage by category. "Fragment" indicates the SMT fragment of generated constraints.

Category	**Count**	**Fragment**
Convolution (Conv1d–3d, ConvTranspose)	12	QF_LIA
Normalization (BatchNorm, LayerNorm, GroupNorm)	15	QF_LIA
Attention (MultiheadAttention, etc.)	4	QF_LIA
Recurrence (LSTM, GRU, RNN)	6	QF_LIA
Pooling (MaxPool, AvgPool, AdaptivePool)	16	QF_LIA
Activation (ReLU, GELU, Sigmoid, etc.)	25	trivial
Padding (ZeroPad, ReflectionPad, etc.)	15	QF_LIA
Loss (CrossEntropy, MSE, NLL, etc.)	21	QF_LIA
Embedding (Embedding, EmbeddingBag)	2	QF_LIA
Structural (reshape, cat, split, transpose)	26	QF_LIA/NIA
Total	**142**	

Table 2: Representative shape transfer functions. $s_{[i]}$ denotes the i-th dimension of shape s.

Operator	**Precondition**	**Output Shape**
$\texttt{Linear}(f_{\text{in}}, f_{\text{out}})$	$s_{[-1]} = f_{\text{in}}$	$s_{[0:-1]} \parallel [f_{\text{out}}]$
$\texttt{Conv2d}(c_{\text{in}}, c_{\text{out}}, k)$	$\|s\| = 4 \wedge s_{[1]} = c_{\text{in}}$	$(s_{[0]}, c_{\text{out}}, s'_H, s'_W)$
$\texttt{BatchNorm}(n)$	$s_{[1]} = n$	s
$\texttt{LayerNorm}(n)$	$s_{[-1]} = n$	s
$\texttt{Reshape}(d_1, \ldots, d_k)$	$\prod s_i = \prod d_j$	$(d_1, \ldots, d_k)$
$\texttt{cat}(t_1, \ldots, t_n; d)$	$\forall i, j.\ s_i^{[\neg d]} = s_j^{[\neg d]}$	$s_1^{[\neg d]} \parallel [\sum_i s_i^{[d]}]$
$\texttt{matmul}(A, B)$	$A_{[-1]} = B_{[-2]}$	$\text{broadcast}(A_{[:-2]}, B_{[:-2]})$ $\parallel [A_{[-2]}, B_{[-1]}]$

5 Operator Transfer Functions

Each of the 142 supported operators has a *transfer function* τ_ℓ mapping input tensor state to output tensor state with a precondition $pre(\tau_\ell, \sigma)$. Transfer functions encode shape transformation rules, device propagation, phase sensitivity, stride effects, and permutation changes.

5.1 Coverage

Table 1 summarizes operator coverage by category.

Of these, 136 produce decidable QF_LIA constraints. Only 6 operators (reshape, view, flatten, PixelShuffle, repeat, unflatten) produce QF_NIA constraints via the product-equality invariant $\prod_i s_i = \prod_j d_j$.

5.2 Representative Transfer Functions

Table 2 shows representative shape transfer functions. We present the full formal typing rules in §6.

Conv2d output dimensions. For Conv2d, the output spatial dimensions are:

$$s'_H = \left\lfloor \frac{s_{[2]} + 2p_H - d_H(k_H - 1) - 1}{st_H} \right\rfloor + 1$$

where p_H is padding, d_H is dilation, k_H is kernel size, and st_H is stride. These parameters are tracked jointly by $\mathcal{T}_{shape}$ and $\mathcal{T}_{stride}$.

Conservative handling of unsupported operators. When TensorGuard encounters an operator outside its 142-operator registry, it marks the output shape as Unknown, propagating an explicit loss-of-information marker through downstream constraints. This ensures that unsupported operators never produce false negatives: verification may become inconclusive (Unknown) but will never incorrectly report safety.

6 Formal Typing Rules

We present the typing rules in the standard judgment form $\Gamma \vdash e : \tau$, where Γ is a typing context mapping variables to refined tensor types and τ is a refined output type.

6.1 Core Rules

T-Linear.

$$\frac{\Gamma \vdash x : \{t : \mathit{Tensor} \mid \mathit{dim}_{-1}(t) = f_{\text{in}}\} \qquad f_{\text{in}}, f_{\text{out}} \in \mathbb{Z}_{>0}}{\Gamma \vdash \texttt{Linear}(f_{\text{in}}, f_{\text{out}})(x) : \{t' : \mathit{Tensor} \mid \mathit{shape}(t') = \mathit{shape}(x)_{[0:-1]} \parallel [f_{\text{out}}]\}} \;\textsc{T-Linear}$$

T-Conv2d.

$$\frac{\Gamma \vdash x : \{t \mid \mathit{ndim}(t) = 4 \land \mathit{dim}_1(t) = c_{\text{in}}\} \qquad k, st, p, d \in \mathbb{Z}_{>0}}{\Gamma \vdash \texttt{Conv2d}(c_{\text{in}}, c_{\text{out}}, k, st, p, d)(x) : \{t' \mid \mathit{dim}_0(t') = \mathit{dim}_0(x) \land \mathit{dim}_1(t') = c_{\text{out}} \land \mathit{dim}_i(t') = \lfloor \frac{\mathit{dim}_i(x) + 2p - d(k-1) - 1}{st} \rfloor + 1}$$

T-Broadcast.

$$\frac{\Gamma \vdash a : \{t \mid \mathit{shape}(t) = s_a\} \Gamma \vdash b : \{t \mid \mathit{shape}(t) = s_b\} \forall i.\ s_a[i] = s_b[i] \lor s_a[i] = 1 \lor s_b[i] = 1}{\Gamma \vdash a \oplus b : \{t' \mid \mathit{dim}_i(t') = \max(s_a[i], s_b[i])\}} \;\textsc{T-Broadcast}$$

T-Reshape.

$$\frac{\Gamma \vdash x : \{t \mid \mathit{shape}(t) = s\} \qquad \prod_i s_i = \prod_j d_j}{\Gamma \vdash \texttt{reshape}(x, d_1, \ldots, d_k) : \{t' \mid \mathit{shape}(t') = (d_1, \ldots, d_k)\}} \;\textsc{T-Reshape}$$

T-Cat.

$$\frac{\Gamma \vdash t_i : \{t \mid \mathit{shape}(t) = s_i\} \quad (i = 1, \ldots, n) \quad \forall i, j.\ \forall k \neq d.\ s_i[k] = s_j[k]}{\Gamma \vdash \texttt{cat}([t_1, \ldots, t_n], d) : \{t' \mid \mathit{dim}_d(t') = \sum_i s_i[d] \land \forall k \neq d.\ \mathit{dim}_k(t') = s_1[k]\}} \;\textsc{T-Cat}$$

T-Matmul.

$$\frac{\Gamma \vdash A : \{t \mid \mathit{shape}(t) = s_A\} \quad \Gamma \vdash B : \{t \mid \mathit{shape}(t) = s_B\} \quad s_A[-1] = s_B[-2]}{\Gamma \vdash \texttt{matmul}(A, B) : \{t' \mid \mathit{shape}(t') = \mathit{broadcast}(s_A[:-2], s_B[:-2]) \parallel [s_A[-2], s_B[-1]]\}} \;\textsc{T-Matmul}$$

6.2 Multi-Theory Rules

The following rules involve constraints across multiple theories.

T-DeviceTransfer.

$$\frac{\Gamma \vdash x : \{t \mid \mathit{dev}(t) = d_1\} \qquad d_2 \in \mathcal{D}}{\Gamma \vdash x\texttt{.to}(d_2) : \{t' \mid \mathit{shape}(t') = \mathit{shape}(x) \land \mathit{dev}(t') = d_2\}} \;\textsc{T-To}$$

T-PhaseConditional.

$$\frac{\Gamma, \mathit{mode} = \texttt{train} \vdash e_1 : \tau_1 \Gamma, \mathit{mode} = \texttt{eval} \vdash e_2 : \tau_2}{\Gamma \vdash \texttt{if self.training:}\ e_1\ \texttt{else:}\ e_2 : \tau_1 \sqcap \tau_2} \;\textsc{T-Phase}$$

Here $\tau_1 \sqcap \tau_2$ denotes the meet (intersection) of the two refined types: both must be well-typed for the model to be safe.

6.3 Additional Operator Rules

T-Conv1d.

$$\frac{\Gamma \vdash x : \{t \mid \mathit{ndim}(t) = 3 \wedge \mathit{dim}_1(t) = c_{\text{in}}\}}{\Gamma \vdash \texttt{Conv1d}(c_{\text{in}}, c_{\text{out}}, k)(x) : \{t' \mid \mathit{dim}_0(t') = \mathit{dim}_0(x) \wedge \mathit{dim}_1(t') = c_{\text{out}} \wedge \mathit{dim}_2(t') = \lfloor \frac{\mathit{dim}_2(x) + 2p - d(k-1) - 1}{st} \rfloor + 1\}} \text{ T-C}$$

T-BatchNorm.

$$\frac{\Gamma \vdash x : \{t \mid \mathit{dim}_1(t) = n\}}{\Gamma \vdash \texttt{BatchNorm}(n)(x) : \{t' \mid \mathit{shape}(t') = \mathit{shape}(x)\}} \text{ T-BN}$$

T-Flatten.

$$\frac{\Gamma \vdash x : \{t \mid \mathit{shape}(t) = s\} \qquad 0 \leq d_s \leq d_e < \mathit{ndim}(x)}{\Gamma \vdash \texttt{flatten}(x, d_s, d_e) : \{t' \mid \mathit{shape}(t') = s_{[0:d_s]} \parallel [\prod_{i=d_s}^{d_e} s_i] \parallel s_{[d_e+1:]}\}} \text{ T-}\textsc{Flatten}$$

T-Transpose.

$$\frac{\Gamma \vdash x : \{t \mid \mathit{shape}(t) = s\} \qquad 0 \leq d_0, d_1 < \mathit{ndim}(x)}{\Gamma \vdash \texttt{transpose}(x, d_0, d_1) : \{t' \mid \mathit{dim}_{d_0}(t') = s_{d_1} \wedge \mathit{dim}_{d_1}(t') = s_{d_0} \wedge \forall k \notin \{d_0, d_1\}.\ \mathit{dim}_k(t') = s_k\}} \text{ T-}\textsc{Transpose}$$

T-MultiheadAttention.

$$\frac{\Gamma \vdash q : \{t \mid \mathit{shape}(t) = (L, N, E)\} \Gamma \vdash k : \{t \mid \mathit{shape}(t) = (S, N, E_k)\} \Gamma \vdash v : \{t \mid \mathit{shape}(t) = (S, N, E_v)\} E \bmod h = 0}{\Gamma \vdash \texttt{MHA}(E, h)(q, k, v) : \{t' \mid \mathit{shape}(t') = (L, N, E)\}} \text{ T-MHA}$$

T-LSTM.

$$\frac{\Gamma \vdash x : \{t \mid \mathit{ndim}(t) = 3 \wedge \mathit{dim}_2(t) = f_{\text{in}}\}}{\Gamma \vdash \texttt{LSTM}(f_{\text{in}}, h, L, bi)(x) : \{(o, (h_n, c_n)) \mid \mathit{dim}_2(o) = h \cdot (1 + bi)\}} \text{ T-LSTM}$$

T-MaxPool2d.

$$\frac{\Gamma \vdash x : \{t \mid \mathit{ndim}(t) = 4\}}{\Gamma \vdash \texttt{MaxPool2d}(k, st, p)(x) : \{t' \mid \mathit{dim}_{0,1}(t') = \mathit{dim}_{0,1}(x) \wedge \forall i \in \{2, 3\}.\ \mathit{dim}_i(t') = \lfloor \frac{\mathit{dim}_i(x) + 2p_i - k_i}{st_i} \rfloor + 1\}} \text{ T-}\textsc{MaxPool2d}$$

T-AdaptiveAvgPool2d.

$$\frac{\Gamma \vdash x : \{t \mid \mathit{ndim}(t) = 4\}}{\Gamma \vdash \texttt{AdaptiveAvgPool2d}(h', w')(x) : \{t' \mid \mathit{dim}_{0,1}(t') = \mathit{dim}_{0,1}(x) \wedge \mathit{dim}_2(t') = h' \wedge \mathit{dim}_3(t') = w'\}} \text{ T-}\textsc{AdaptPool}$$

T-Split.

$$\frac{\Gamma \vdash x : \{t \mid \mathit{shape}(t) = s\} \qquad s_d \bmod n = 0}{\Gamma \vdash \texttt{split}(x, s_d/n, d) : [t'_1, \ldots, t'_n] \text{ where } \forall i.\ \mathit{dim}_d(t'_i) = s_d/n} \text{ T-}\textsc{Split}$$

6.4 Soundness

Conjecture 6.1 (Type Soundness). *If $\Gamma \vdash e : \tau$ is derivable and $\sigma \models \Gamma$, then evaluation of e under σ does not produce a shape, device, phase, stride, or permutation error, and the result satisfies the refinement predicate of τ.*

Algorithm 2 IC3/PDR for parametric shape safety

Require: Shape transition system (I, T, P)
Ensure: SAFE (with inductive invariant) or UNSAFE (with trace)

```
F_0 ← I; k ← 1
loop
    while ∃ cube c s.t. F_{k−1} ∧ T ∧ ¬P ∧ c is SAT do
        Block c in F_{k−1} by generalization
    end while
    F_k ← F_{k−1} \ blocked cubes
    if F_k = F_{k−1} then return SAFE with inductive invariant F_k
    end if
    k ← k + 1
end loop
```

Proof sketch. By induction on the typing derivation. Each typing rule's precondition exactly encodes the compatibility check that PyTorch performs at runtime, and the postcondition faithfully models the output shape computation documented in the PyTorch operator specifications. Soundness of $\mathcal{T}_\Pi$ (Theorem 3.8) ensures cross-theory constraints are handled correctly. The gap between this sketch and a full proof lies in (1) faithful modeling of all 142 operator semantics and (2) conformance between the Lean specification and the Python implementation, which is validated by property-based testing (Hypothesis) against PyTorch runtime semantics. □

7 IC3/PDR Parametric Verification

Bounded verification checks safety for concrete input dimensions. TensorGuard's IC3/PDR engine goes further: it proves safety for *all* valid input dimensions simultaneously, yielding parametric safety certificates.

7.1 Encoding as a Transition System

We cast tensor shape verification as a safety property over the Kripke structure induced by the computation graph.

Definition 7.1 (Shape Transition System)**.** A *shape transition system* (I, T, P) consists of:

- Initial states I: shape assignments satisfying input constraints (e.g., $ndim = 4$, $dim_1 = 3$).
- Transition relation T: each layer's transfer function τ_ℓ defines a transition from input shape to output shape.
- Safety property P: the conjunction of all operator preconditions.

7.2 IC3 Algorithm for Shapes

IC3/PDR [1, 2] discovers inductive invariants without unrolling the transition relation. It maintains a sequence of frames $F_0, F_1, \ldots, F_N$ such that $I \subseteq F_0$ and each F_i overapproximates the states reachable in $\leq i$ steps. If some $F_i = F_{i+1}$, the property holds inductively for all reachable states.

Key insight. For typical neural network computation graphs, the IC3 frames converge rapidly—in our experiments, at most 3 frames regardless of network depth (50+ layers). This is because tensor shape transformations are predominantly affine, and affine transition systems have short inductive invariants.

Completeness. IC3 is complete for QF_LIA transition systems (136 of 142 operators). For the 6 QF_NIA operators (reshape family), the procedure is sound but incomplete: it may fail to find an inductive invariant even when one exists.

7.3 SSA Encoding

The shape transition system is encoded in Static Single Assignment (SSA) form. Each tensor variable t_i at computation graph node i is assigned exactly once, producing a flat conjunction of transition constraints:

$$T \;=\; \bigwedge_{i=1}^{n} \big(\sigma_i = \tau_{\ell_i}(\sigma_{i-1}) \;\wedge\; \mathit{pre}(\tau_{\ell_i}, \sigma_{i-1})\big)$$

where σ_i is the tensor metadata state after the i-th operation and ℓ_i is the operator at node i. The SSA encoding eliminates the need for explicit frame variables and simplifies the IC3 generalization step, because cube literals refer to individual SSA variables rather than mutable state.

Cube generalization. When IC3 blocks a cube c (a conjunction of literals describing a bad state), it attempts to generalize c by dropping literals while maintaining the blocking property. In the shape domain, this corresponds to discovering that a safety violation is independent of certain dimension values. For example, if a matmul precondition $A_{[-1]} = B_{[-2]}$ is violated, the blocking cube may be generalized to drop all constraints on batch dimensions, yielding a more general invariant.

Inductive invariant extraction. When IC3 terminates with $F_k = F_{k+1}$, the invariant F_k is a conjunction of shape constraints that hold at every reachable state. This invariant serves as the proof certificate: it can be independently verified by checking that (1) $I \subseteq F_k$, (2) $F_k \wedge T \subseteq F'_k$ (inductiveness), and (3) $F_k \subseteq P$ (safety). The certificate is a self-contained artifact that does not depend on the IC3 search history.

7.4 Parametric Certificates

A parametric safety certificate asserts that a model is safe for *all* valid input shapes simultaneously:

Definition 7.2 (Parametric Safety Certificate)**.** A *parametric safety certificate* for a computation graph G with symbolic input shape $(d_1, d_2, \ldots, d_r)$ is a formula $\mathit{Cert}(d_1, \ldots, d_r)$ such that:

$$\forall d_1, \ldots, d_r \in \mathbb{Z}_{\geq 1}.\ \mathit{Cert}(d_1, \ldots, d_r) \implies \mathit{Safe}(G, d_1, \ldots, d_r)$$

and *Cert* is the inductive invariant discovered by IC3.

For example, verifying a ResNet with symbolic batch size B, height H, and width W produces a certificate asserting safety for all $B, H, W \geq 1$, provided H and W are compatible with the network's downsampling factors. This is strictly stronger than testing with concrete dimensions.

8 Formal Properties and Metatheory

This section develops the formal metatheoretic properties of TensorGuard's type system and verification engine, including decidability results, complexity analysis, and completeness characterizations.

8.1 Decidability of Verification Conditions

Theorem 8.1 (Decidability of Core Verification)**.** *For any computation graph G composed entirely of operators generating QF_LIA constraints (136 of 142 supported operators), the verification condition $\mathit{Init} \wedge \mathit{VC} \wedge \neg \mathit{Safe}$ is decidable.*

Proof. QF_LIA is a decidable fragment of first-order arithmetic. The verification condition is a conjunction of QF_LIA formulas (one per operator), and conjunctions of QF_LIA formulas are QF_LIA. Z3's Simplex-based decision procedure terminates on all QF_LIA inputs [8]. □

Theorem 8.2 (Semi-Decidability of Reshape Verification)**.** *For computation graphs containing reshape operators, the verification condition falls in QF_NIA, which is undecidable in general. However, the restricted NIA subfragment generated by* TensorGuard *(at most one symbolic factor per product term) is decidable.*

Proof. The reshape constraint $\prod_i s_i = \prod_j d_j$ where each s_i, d_j is either a constant or a single symbolic variable falls in the semi-linear fragment of integer arithmetic. This fragment can be decided by encoding product terms as sums via logarithmic relaxation, or by case-splitting on the signs and magnitudes of symbolic variables. Z3's nonlinear extension of Simplex handles this restricted fragment reliably via incremental linearization [8]. □

8.2 Complexity Analysis

Theorem 8.3 (Verification Complexity)**.** *Let G be a computation graph with n operators and k shared variables across theories. The worst-case complexity of* TensorGuard*'s verification is:*

$$O\Big(n \cdot p(n) + S(k_d, |\mathcal{D}|) \cdot S(k_p, |\mathcal{P}|) \cdot q(n)\Big)$$

where $p(n)$ is the cost of constraint generation (linear in the number of operators), $q(n)$ is the cost of a single Z3 call, and $S(k, n)$ is the Stirling number of the second kind.

Proof. The verification procedure has two phases:

1. **Constraint generation**: traverses the computation graph in topological order, generating QF_LIA/NIA constraints for each of the n operators. Cost: $O(n \cdot p(n))$.
2. **Theory combination**: enumerates arrangements of shared variables. For k_d device variables over domain $\mathcal{D}$ and k_p phase variables over $\mathcal{P}$, the number of arrangements is $S(k_d, |\mathcal{D}|) \cdot S(k_p, |\mathcal{P}|)$. Each arrangement requires one Z3 call.

In practice, $k_d \leq 4$ and $k_p \leq 2$, yielding at most $15 \cdot 1 = 15$ arrangements, so the theory combination overhead is a small constant factor. □

Corollary 8.4 (Practical Linear-Time Behavior)**.** *For models with bounded shared variables (which includes all 108 models in our evaluation), the verification time is dominated by constraint generation and a single Z3 call, yielding effectively linear time in the number of operators.*

8.3 Completeness Characterization

TensorGuard is designed to be sound but not complete. We characterize the sources of incompleteness precisely.

Definition 8.5 (Completeness Gap)**.** The *completeness gap* of TensorGuard is the set of computation graphs G such that G has a runtime error but TensorGuard reports Unknown or Safe.

Theorem 8.6 (Completeness Gap Characterization)**.** *The completeness gap arises from exactly three sources:*

1. ***Unsupported operators**: graphs containing operators outside the 142-operator registry.*
2. ***Data-dependent shapes**: graphs where tensor shapes depend on tensor* values *(e.g., `torch.nonzero`).*
3. ***Dynamic control flow**: graphs with data-dependent iteration counts or recursive module calls.*

For computation graphs composed entirely of supported operators with data-independent control flow, TensorGuard *is complete for shape, device, and phase errors.*

Proof sketch. For supported operators with data-independent control flow, the computation graph is fully extracted and each operator's transfer function faithfully models the precondition and postcondition. The SMT encoding is exact (not an overapproximation) for QF_LIA constraints, and the restricted QF_NIA fragment admits exact solutions. Unsatisfiability of the safety negation is therefore equivalent to the absence of runtime errors.

The three sources of incompleteness are:

1. Unsupported operators produce Unknown output shapes, which conservatively block error detection downstream.
2. Data-dependent shapes introduce quantifier alternation (existential over input values, universal over resulting shapes) that falls outside the quantifier-free fragment.
3. Dynamic control flow requires loop invariant inference, which is undecidable in general (Rice's theorem [14]).

□

8.4 Soundness of Multi-Phase Verification

Theorem 8.7 (Multi-Phase Soundness)**.** *If* TensorGuard *reports* Safe *for a model* M*, then* M *does not produce shape, device, or phase errors in either training mode (*`self.training` $=$ `True`*) or eval mode (*`self.training` $=$ `False`*).*

Proof. TensorGuard's multi-phase verification verifies both execution branches:

1. The `training` branch is verified with $mode = $ `train`, which sets `self.training` $=$ `True` and includes stochastic operators (Dropout, DropPath).
2. The `eval` branch is verified with $mode = $ `eval`, which sets `self.training` $=$ `False` and disables stochastic operators.

If either branch contains a safety violation, TensorGuard reports Unsafe with the violating phase annotated in the counterexample. Safe is returned only when both branches pass verification.

The key insight is that conditional branches on `self.training` are *statically resolved*: the graph extractor produces two separate subgraphs rather than a single graph with a conditional. This avoids the need for predicate abstraction or path-sensitive analysis. □

8.5 Monotonicity of the Operator Lattice

Definition 8.8 (Operator Monotonicity)**.** An operator ℓ is *monotone* with respect to the shape ordering $\leq_s$ if for all shapes $s_1 \leq_s s_2$:

$$pre(\ell, s_1) \implies pre(\ell, s_2) \quad \text{and} \quad \ell(s_1) \leq_s \ell(s_2)$$

Proposition 8.9 (Monotonicity of Core Operators)**.** *All 25 activation operators, all 15 normalization operators (with fixed parameters), and all 15 padding operators are monotone. The 12 convolution operators are monotone in batch and spatial dimensions but not in channel dimensions.*

Proof. Activation operators are shape-preserving ($\ell(s) = s$), so monotonicity is trivial. Normalization operators check a specific dimension against a fixed parameter; if the precondition holds for s_1, it holds for any s_2 with the same value at that dimension. Padding operators add fixed amounts to spatial dimensions, which is monotone. Convolution operators apply an affine transformation to spatial dimensions (monotone) but require a fixed channel dimension (not monotone in channels). □

8.6 Fixpoint Characterization of IC3 Convergence

Theorem 8.10 (IC3 Frame Convergence for Affine Systems)**.** *For a shape transition system* (I, T, P) *where all transitions are affine (i.e.,* $T(\sigma) = A\sigma + b$ *for matrix* A *and vector* b*), IC3 converges in at most* $rank(A - I) + 1$ *frames.*

Proof sketch. Each IC3 frame F_k overapproximates the states reachable in $\leq k$ steps. For affine transitions, the reachable sets stabilize after at most $rank(A - I)$ iterations (the dimension of the "non-fixed-point" subspace). Since neural network layers are predominantly affine (convolution, linear, normalization), and the eigenvalues of the induced shape transformation matrix A are all 0 or 1 (shape dimensions either change by a fixed amount or are preserved), $rank(A - I)$ is small (typically 1–2 for spatial dimension changes and 1 for channel changes).

In our experiments, all 108 models converge in at most 3 frames, consistent with this analysis: the spatial, channel, and batch dimension "modes" of the shape transformation each contribute at most one frame. □

8.7 Relationship to Dependent Type Theory

TensorGuard's refinement types can be viewed as a restricted form of dependent types where the index domain is limited to tensor metadata.

Proposition 8.11 (Embedding in Dependent Types)**.** TensorGuard*'s type system can be faithfully embedded in a dependent type system with refinement sorts, such as F* [4] or Liquid Types [3], by encoding:*

- *Tensor types as dependent records:* $\texttt{Tensor}(s, d, p, str, \pi) : \texttt{Type}$

- *Shape constraints as refinement predicates:* $\{t : \textit{\texttt{Tensor}} \mid \mathit{dim}_{-1}(t) = 128\}$
- *Operator types as dependent function types:* $\textit{\texttt{Linear}} : (x : \{t \mid \mathit{dim}_{-1}(t) = f_{in}\}) \to \{t' \mid \mathit{dim}_{-1}(t') = f_{out}\}$

The restriction to tensor metadata (shapes, devices, phases, strides, permutations) rather than arbitrary program values is what makes TensorGuard's type checking decidable: the refinement predicates are always in a decidable fragment (QF_LIA or restricted QF_NIA), unlike general dependent types where type checking is undecidable.

This design choice—restricting the refinement domain to a decidable fragment while maximizing expressiveness within that fragment—is the key architectural contribution of TensorGuard.

9 Denotational Semantics

We give a denotational semantics to TensorGuard's type system, showing that refinement types denote sets of tensor values and that subtyping corresponds to set inclusion.

9.1 Semantic Domains

Definition 9.1 (Tensor Value)**.** A *tensor value* v is a tuple $(A, s, d, p, \mathit{str}, \pi)$ where:
- A is the underlying data array
- $s \in \mathbb{Z}^*_{\geq 1}$ is the shape tuple
- $d \in \mathcal{D}$ is the device placement
- $p \in \mathcal{P}$ is the execution phase context
- $\mathit{str} \in \mathbb{Z}^{|s|}_{\geq 1}$ is the stride tuple
- $\pi \in S_{|s|}$ is the axis permutation

Definition 9.2 (Type Denotation)**.** The denotation of a refined tensor type $\tau = \{t : \mathit{Tensor} \mid \varphi\}$ is:

$$[\![\tau]\!] = \{v \mid v \text{ is a tensor value and } v \models \varphi\}$$

where $v \models \varphi$ means the predicate φ holds when its free variables are interpreted over v's metadata.

Proposition 9.3 (Subtyping Soundness)**.** $\tau_1 \leq \tau_2$ *iff* $[\![\tau_1]\!] \subseteq [\![\tau_2]\!]$.

Proof. Let $\tau_1 = \{t \mid \varphi_1\}$ and $\tau_2 = \{t \mid \varphi_2\}$. Then $\tau_1 \leq \tau_2$ iff $\varphi_1 \Rightarrow \varphi_2$ is valid (by the subtyping rule), iff for all tensor values v, $v \models \varphi_1$ implies $v \models \varphi_2$, iff $[\![\tau_1]\!] \subseteq [\![\tau_2]\!]$. ☐

9.2 Operator Semantics

Each operator ℓ has both an *abstract semantics* (the transfer function τ_ℓ operating on refinement types) and a *concrete semantics* (the PyTorch runtime behavior $[\![\ell]\!]$ operating on tensor values). Soundness requires that the abstract semantics overapproximates the concrete:

Definition 9.4 (Transfer Function Soundness)**.** A transfer function τ_ℓ is *sound* with respect to concrete semantics $[\![\ell]\!]$ if for all tensor values v:

$$v \in [\![\mathit{dom}(\tau_\ell)]\!] \implies [\![\ell]\!](v) \in [\![\mathit{cod}(\tau_\ell)(v)]\!]$$

where *dom* and *cod* are the domain and codomain types of τ_ℓ.

Example 9.5 (Linear Soundness)**.** For $\texttt{Linear}(f_{\text{in}}, f_{\text{out}})$:
- Domain: $\{t \mid \mathit{dim}_{-1}(t) = f_{\text{in}}\}$
- Codomain: $\{t' \mid \mathit{shape}(t') = \mathit{shape}(t)_{[0:-1]} \| [f_{\text{out}}]\}$
- Concrete: $[\![\texttt{Linear}]\!](x) = xW^T + b$ where $W \in \mathbb{R}^{f_{\text{out}} \times f_{\text{in}}}$
- Soundness: if x has last dimension f_{in}, the matrix product xW^T produces last dimension f_{out}, matching the codomain predicate.

Algorithm 3 Restricted growth string enumeration

Require: k elements, maximum m classes
Ensure: All restricted growth strings

$a \leftarrow [0, 0, \ldots, 0]$ ▷ length k
yield a
loop
  $i \leftarrow k$ ▷ find rightmost incrementable
  while $i > 0$ and $a[i] \geq \min(\max(a[1..i-1]) + 1, m-1)$ **do**
    $i \leftarrow i - 1$
  end while
  if $i = 0$ **then return** ▷ exhausted
  end if
  $a[i] \leftarrow a[i] + 1$
  for $j = i + 1$ to k **do**
    $a[j] \leftarrow 0$
  end for
  yield a
end loop

9.3 Compositional Semantics

The semantics of a computation graph $G = (V, E, \lambda)$ is the composition of individual operator semantics along the edges:

$$[\![G]\!] = [\![\ell_n]\!] \circ [\![\ell_{n-1}]\!] \circ \cdots \circ [\![\ell_1]\!]$$

where $\ell_1, \ldots, \ell_n$ is a topological ordering of the operators. The verification condition is sound because transfer function soundness composes:

Theorem 9.6 (Compositional Soundness)**.** *If each transfer function τ_{ℓ_i} is sound, then the composed verification condition is sound: if $Init \wedge VC \wedge \neg Safe$ is unsatisfiable, then no input satisfying Init produces a runtime error when processed by G.*

Proof. By induction on the topological ordering. At each step i, the induction hypothesis guarantees that the tensor metadata at node i satisfies the postcondition of τ_{ℓ_i}. This postcondition is the precondition of $\tau_{\ell_{i+1}}$, so the composition preserves soundness. The conjunction of all preconditions is exactly *Safe*, so unsatisfiability of $Init \wedge VC \wedge \neg Safe$ implies no precondition violation. □

10 Extended Algorithm Details

10.1 Arrangement Enumeration

The Tinelli–Zarba procedure enumerates arrangements of shared variables over finite-domain sorts. An arrangement assigns shared variables to equivalence classes, with the number of classes bounded by the domain cardinality.

Definition 10.1 (Restricted Growth String)**.** A *restricted growth string* of length k with maximum m is a sequence $a_1, \ldots, a_k$ where:

- $a_1 = 0$
- $a_{i+1} \leq \max(a_1, \ldots, a_i) + 1$
- $\max(a_1, \ldots, a_k) < m$

Each restricted growth string corresponds to a unique partition of $\{1, \ldots, k\}$ into at most m equivalence classes.

Algorithm 4 Nelson–Oppen equality propagation

Require: Solvers $S_1, \ldots, S_n$ with shared variables V
Ensure: Fixed-point equality set E

$E \leftarrow \emptyset$; *changed* $\leftarrow$ **true**
while *changed* **do**
    changed $\leftarrow$ **false**
    for each solver S_i **do**
        for each pair $(v_a, v_b) \in V \times V \setminus E$ **do**
            if $S_i \models v_a = v_b$ **then**
                $E \leftarrow E \cup \{(v_a, v_b)\}$
                Assert $v_a = v_b$ in all other solvers
                changed $\leftarrow$ **true**
            end if
        end for
    end for
end whilereturn E

Arrangement-to-constraint translation. Given an arrangement α (a restricted growth string), we translate it to a conjunction of equalities and disequalities over shared variables:

$$constr(\alpha) = \bigwedge_{i<j, \alpha_i = \alpha_j} (v_i = v_j) \ \wedge \bigwedge_{i<j, \alpha_i \neq \alpha_j} (v_i \neq v_j)$$

This conjunction is asserted in each theory solver before checking satisfiability.

10.2 Nelson–Oppen Equality Propagation

For stably-infinite theories ($\mathcal{T}_{shape}$, $\mathcal{T}_{stride}$, $\mathcal{T}_{perm}$), equalities over shared variables are propagated via the Nelson–Oppen deduction-propagation loop:

Termination. The loop terminates because each iteration either adds an equality to E or terminates, and $|E|$ is bounded by $|V|^2$. In practice, the number of shared variables between shape and stride theories is small (typically ≤ 8), so the loop converges in 1–2 iterations.

10.3 Graph Extraction Algorithm

The AST-based graph extractor is the most commonly used backend. It operates in two passes:

10.4 Constraint Generation Details

The constraint generator translates each edge in the computation graph to SMT formulas. We describe the encoding for each theory.

Shape constraints. For each operator with precondition *pre* and output shape function f, we generate:

$$\begin{aligned} pre(\sigma_{in}) & \quad \text{(precondition)} \\ \sigma_{out} = f(\sigma_{in}) & \quad \text{(transfer)} \end{aligned}$$

where σ_{in} and σ_{out} are tuples of Z3 integer variables representing input and output shape dimensions.

Device constraints. For each binary operator:

$$dev(\sigma_{in,1}) = dev(\sigma_{in,2})$$

Device variables are Z3 integer constants in $\{0, 1, 2, 3, 4\}$ representing $\{\texttt{cpu}, \texttt{cuda:0}, \ldots, \texttt{cuda:3}\}$.

Algorithm 5 AST-based graph extraction

Require: Python source code of an `nn.Module`
Ensure: Computation graph $G = (V, E, \lambda)$

Parse source into Python AST T
▷ Pass 1: Extract layer declarations from __init__
$\mathit{layers} \leftarrow \emptyset$
for each `self.X = nn.Y(...)` in T.`__init__` **do**
  Match `Y` against 142-operator registry
  if matched **then**
    $\mathit{layers}[\mathtt{X}] \leftarrow (\mathtt{Y}, \mathit{params})$
  else
    $\mathit{layers}[\mathtt{X}] \leftarrow$ UNKNOWN
  end if
end for
▷ Pass 2: Trace data flow in forward
$G \leftarrow$ empty graph; $\mathit{cur} \leftarrow$ input node
for each statement s in T.`forward` **do**
  if s is `x = self.Y(x)` and `Y` $\in \mathit{layers}$ **then**
    Add edge $(\mathit{cur}, \mathit{new}, \mathit{layers}[\mathtt{Y}])$
    $\mathit{cur} \leftarrow \mathit{new}$
  else if s is `if self.training:` **then**
    Fork graph into train/eval branches
    Recursively trace both branches
  else
    Pattern-match tensor operations (view, cat, etc.)
  end if
end forreturn G

Phase constraints. For each phase-conditional branch:

$$\mathit{mode} = 0 \implies VC_{train}$$
$$\mathit{mode} = 1 \implies VC_{eval}$$

Both implications must hold, corresponding to verifying both branches.

Stride constraints. For operations sensitive to contiguity:

$$\neg \mathit{contiguous}(\sigma_{in}) \implies \textsc{Error}(\mathtt{view})$$

where contiguity is encoded as the stride formula from §3.5.

Permutation constraints. For transpose operations:

$$\sigma_{out} = \mathsf{swap}(\sigma_{in}, d_0, d_1)$$

with the involution axiom (Equation 3) available for reasoning about sequences of transpositions.

11 Extended Examples

We present additional cross-cutting bug examples to illustrate TENSORGUARD's multi-theory reasoning in detail.

11.1 Example: Feature Pyramid Network Channel Mismatch

```python
1  class FPNBug(nn.Module):
2      def __init__(self):
3          super().__init__()
4          self.backbone_c3 = nn.Conv2d(3, 128, 3, padding=1)
5          self.backbone_c4 = nn.Conv2d(128, 256, 3, stride=2,
6                                        padding=1)
7          self.backbone_c5 = nn.Conv2d(256, 512, 3, stride=2,
8                                        padding=1)
9          # BUG: lateral_c4 expects 128 channels but gets 256
10         self.lateral_c4 = nn.Conv2d(128, 256, 1)
11         self.lateral_c5 = nn.Conv2d(512, 256, 1)
12
13     def forward(self, x):
14         c3 = self.backbone_c3(x)
15         c4 = self.backbone_c4(c3)
16         c5 = self.backbone_c5(c4)
17         p5 = self.lateral_c5(c5)
18         # lateral_c4 gets c4 (256 channels), expects 128
19         p4 = self.lateral_c4(c4) + F.interpolate(
20             p5, scale_factor=2)
21         return p4
```

Listing 4: FPN lateral connection bug from torchvision-style code. The lateral projection has mismatched input channels.

TensorGuard's trace for this model:

1. Input: shape $(batch, 3, H, W)$, device `cuda:0`
2. `backbone_c3`: $(B, 3, H, W) \to (B, 128, H, W)$. Precondition $dim_1 = 3$: ✓.
3. `backbone_c4`: $(B, 128, H, W) \to (B, 256, H', W')$. Precondition $dim_1 = 128$: ✓.
4. `backbone_c5`: $(B, 256, H', W') \to (B, 512, H'', W'')$. Precondition $dim_1 = 256$: ✓.
5. `lateral_c5`: $(B, 512, H'', W'') \to (B, 256, H'', W'')$. Precondition $dim_1 = 512$: ✓.
6. `lateral_c4`: receives c_4 with $dim_1 = 256$. Precondition $dim_1 = 128$: **FAIL**.

TensorGuard reports:

```
UNSAFE at step 6:
  lateral_c4 expects in_channels=128
  but receives tensor with 256 channels
  Source: self.lateral_c4(c4) at line 16
```

11.2 Example: Squeeze-Excite Block After Transfer Learning

```python
class SEBlockBug(nn.Module):
    def __init__(self, channels=512, reduction=16):
        super().__init__()
        self.conv = nn.Conv2d(3, channels, 3, padding=1)
        self.pool = nn.AdaptiveAvgPool2d(1)
        # BUG: SE block was designed for channels=256
        self.se_fc1 = nn.Linear(256, 256 // reduction)
        self.se_fc2 = nn.Linear(256 // reduction, 256)
        self.relu = nn.ReLU()
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        out = self.conv(x)
        se = self.pool(out).view(out.size(0), -1)
```

```
        se = self.relu(self.se_fc1(se))  # BUG: 512 != 256
        se = self.sigmoid(self.se_fc2(se))
        return out * se.unsqueeze(-1).unsqueeze(-1)
```

Listing 5: Squeeze-excite channel mismatch after changing backbone width. Modeled after timm patterns.

This bug arises during transfer learning: the backbone width was changed from 256 to 512, but the squeeze-excite block was not updated. The mismatch is purely in shape, but in a real-world setting the model might also involve device placement (the SE block on a different GPU) and phase dependence (SE disabled during evaluation in some architectures).

11.3 Example: Anchor Generator Triple-Theory Bug

```
class AnchorGeneratorBug(nn.Module):
    def __init__(self):
        super().__init__()
        self.backbone = nn.Conv2d(3, 256, 3, padding=1)
        self.cls_head = nn.Conv2d(256, 9 * 20, 1)
        # BUG 1: Wrong spatial dims (8x8 vs feature map)
        # BUG 2: Stays on CPU
        # BUG 3: Only used in eval (post-processing)
        self.register_buffer('anchors',
            torch.randn(1, 9, 8, 8, 4))

    def forward(self, x):
        feat = self.backbone(x)
        cls = self.cls_head(feat)
        if not self.training:
            B, C, H, W = cls.shape
            cls = cls.view(B, 9, 20, H, W)
            # Triple bug: shape + device + phase
            anchors = self.anchors[:, :, :H, :W, :]
            output = torch.cat([cls, anchors], dim=2)
        return cls
```

Listing 6: Anchor generator with three simultaneous property violations. Modeled after mmdetection patterns.

This model has three simultaneous property violations:

1. **Shape**: The anchor buffer has spatial dimensions 8×8, but the feature map may have different dimensions depending on input size. The slice `[:H, :W]` may exceed the buffer dimensions.
2. **Device**: `register_buffer` with a `torch.randn` tensor starts on CPU; after `model.cuda()`, it may not migrate.
3. **Phase**: The entire post-processing block is dead during training, so training tests never exercise this code.

TensorGuard detects this via the product domain $\mathcal{T}_{shape} \times \mathcal{T}_{device} \times \mathcal{T}_{phase}$, reporting all three violations in a single verification pass.

11.4 Example: Correct Multi-Theory Model

To demonstrate precision, we show a model that uses multiple theories correctly:

```
class CorrectMultiTheory(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv = nn.Conv2d(3, 64, 3, padding=1)
        self.bn = nn.BatchNorm2d(64)
        self.fc = nn.Linear(64, 10)
```

```python
        self.pool = nn.AdaptiveAvgPool2d(1)
        self.dropout = nn.Dropout(0.5)

    def forward(self, x):
        x = F.relu(self.bn(self.conv(x)))
        x = self.pool(x)
        x = x.view(x.size(0), -1)  # (B, 64)
        if self.training:
            x = self.dropout(x)  # shape-preserving
        return self.fc(x)  # (B, 64) -> (B, 10)
```

Listing 7: Correct model using device transfer, phase-conditional behavior, and complex shape transformations.

TensorGuard verifies this model as safe in both phases:

- Train: Dropout preserves shape; `fc` input is 64.
- Eval: Dropout is inactive; `fc` input is still 64.

No false positive is produced.

12 Evaluation

We evaluate TensorGuard on two axes: (1) cross-cutting multi-theory bug detection—the capability no other tool provides—and (2) real-world model verification. All experiments were conducted on an Apple M-series processor with 16 GB RAM.

12.1 Cross-Cutting Multi-Theory Bugs

Benchmark construction. We construct a benchmark of 52 models (44 buggy, 8 correct) derived from real-world bug patterns. Each buggy model requires reasoning across at least two of the five theories to detect. Sources include:

- **HuggingFace Transformers**: position_ids buffer device mismatch (issue #13666)
- **torchvision**: ResNet bottleneck downsample dimension error, FPN channel mismatch
- **detectron2**: mask head phase-dependent channel mismatch
- **YOLOv5**: anchor grid triple-theory bug (shape + device + eval-only)
- **mmdetection**: anchor generator triple-theory bug
- **timm**: squeeze-excite channel scale after transfer learning
- **fairseq**: label smoothing projection dimension error (training-only)
- **wav2vec2**: feature normalization buffer size mismatch
- **PyTorch Forums / StackOverflow**: BatchNorm channel mismatch, attention bias device errors

Benchmark categories. The 44 buggy models span four cross-cutting categories:

- **Device + Shape** (16 models): device transfers causing shape issues
- **Phase + Shape** (14 models): train/eval mode differences causing shape bugs
- **Device + Phase** (4 models): device-phase interactions
- **Triple-theory** (10 models): shape + device + phase simultaneously

The 8 correct models exercise multiple theories without bugs, testing precision.

Results. Table 3 shows the headline result: TensorGuard achieves F1 = 0.625 with perfect precision, while no baseline tool is designed to detect cross-cutting multi-theory bugs (device × shape, phase × shape). The F1 = 0.000 scores reflect that these tools lack device and phase checking capabilities by design, not that they were run and failed on each benchmark individually. This is not a marginal improvement—it is a categorically new capability.

Table 3: Cross-cutting multi-theory bug detection (52 models). TensorGuard is the only tool with non-zero recall.

Tool	Precision	Recall	F1	FP
TensorGuard (ours)	**1.000**	**0.455**	**0.625**	0
TorchScript†	0.000	0.000	0.000	0
mypy + torch stubs†	0.000	0.000	0.000	0
PyTEA†	0.000	0.000	0.000	0
jaxtyping†	0.000	0.000	0.000	0

†Assessed from documented capabilities; these tools do not check device placement, execution phase, or multi-theory interactions, so they cannot detect these bug classes by design.

Table 4: Per-category cross-cutting results (44 buggy models).

Category	n	TP	FN	F1	Theories
Phase + Shape	14	9	5	0.783	$\mathcal{T}_{phase}, \mathcal{T}_{shape}$
Triple-theory	10	6	4	0.750	$\mathcal{T}_{shape}, \mathcal{T}_{device}, \mathcal{T}_{phase}$
Device + Shape	16	4	12	0.400	$\mathcal{T}_{device}, \mathcal{T}_{shape}$
Device + Phase	4	1	3	0.400	$\mathcal{T}_{device}, \mathcal{T}_{phase}$
Correct (no bug)	8	–	–	1.000	(precision)

Per-category breakdown. Table 4 shows that TensorGuard is strongest on phase-dependent shape bugs ($\mathrm{F1} = 0.783$), where multi-phase verification is essential. The weaker performance on device-only bugs ($\mathrm{F1} = 0.400$) reflects a known limitation: the current device theory catches device issues when they co-occur with shape mismatches, but pure cross-device operations without shape errors are sometimes missed.

12.2 Real-World Architecture Verification

We evaluate TensorGuard on a 56-model real-world architecture suite spanning convolutional, transformer, and hybrid designs, drawn from common PyTorch patterns (ResNet, VGG, BERT, GPT, U-Net, DCGAN, ViT).

TensorGuard achieves $\mathrm{F1} = 0.889$ with 4 false positives (Table 5). Verification completes in under 120 ms per model. The false positives arise from overapproximation in broadcast and reshape analysis; the false negatives arise from models using operators outside the 142-operator registry (marked Unknown, never silently passed).

12.3 Shape-Only Bug Detection

To compare against baselines that operate in the shape-only domain, we evaluate on an 18-model suite of single-theory shape bugs.

Even on single-theory bugs where baselines can compete, TensorGuard achieves the highest F1 (Table 6). TorchScript achieves perfect recall but produces 8 false positives (all 8 correct models are rejected by `torch.jit.script`). The single TensorGuard false positive arises from a `view` with 4 symbolic dimensions where the product-equality constraint is overapproximated.

12.4 Baseline Capability Comparison

Table 7 summarizes the qualitative differences. The unique rows—device checking, phase-aware verification, parametric certificates—correspond directly to TensorGuard's ability to detect cross-cutting bugs that no other tool can find.

Table 5: Real-world architecture verification (56 models).

Metric	Value
Total models	56
Buggy models	36
Correct models	20
Precision	0.889
Recall	0.889
F1	**0.889**
False positives	4
Avg. verification time	91 ms

Table 6: Shape-only bug detection (18 models: 10 buggy, 8 correct).

Tool	Precision	Recall	F1	FP
TensorGuard	0.900	0.900	**0.900**	1
TorchScript	0.556	1.000	0.714	8
mypy	0.000	0.000	0.000	0

12.5 Detailed Verification Traces

To make TensorGuard's reasoning concrete, we present detailed verification traces for representative models from the cross-cutting benchmark.

Trace 1: Phase-dependent shape bug (F1=0.783 category). Consider the transfer learning model from Listing 1. TensorGuard proceeds as follows:

1. **Graph extraction.** AST analysis identifies three layers (`backbone`, `train_head`, `eval_head`) and a phase-conditional branch. Two subgraphs are produced: G_{train} = backbone $\rightarrow$ train_head and G_{eval} = backbone $\rightarrow$ eval_head.
2. **Training branch.** Input: $(B, 512)$. After backbone: $(B, 256)$. Train_head precondition: $dim_{-1} = 256$. Z3 checks $256 = 256$: SAT (precondition holds). Training branch is safe.
3. **Eval branch.** Input: $(B, 512)$. After backbone: $(B, 256)$. Eval_head precondition: $dim_{-1} = 128$. Z3 checks $256 = 128$: UNSAT. Safety negation $\mathit{Init} \wedge \mathit{VC} \wedge \neg \mathit{Safe}$ is SAT with counterexample $B = 1$, producing:

```
UNSAFE [eval] at eval_head:
  expects in_features=128, got 256
```

Trace 2: Triple-theory bug (F1=0.750 category). For the YOLOv5 anchor grid model (Listing 3), TensorGuard activates all three theories:

1. $\mathcal{T}_{phase}$: The post-processing block is gated by `not self.training`, so it is verified only under $mode =$ `eval`.
2. $\mathcal{T}_{shape}$: The anchor buffer has fixed spatial dimensions $(10, 10)$. The feature map spatial dimensions depend on input size. For input $(B, 3, 416, 416)$ with stride-16 backbone: $H = 26, W = 26 \neq 10$. The slice `anchors[:, :, :26, :26, :]` exceeds the buffer dimension.
3. $\mathcal{T}_{device}$: The buffer is created via `torch.randn` (CPU). After `model.cuda()`, the feature map is on GPU. The `torch.cat` operation has a device mismatch precondition violation.
4. Combined: all three violations are reported in a single pass, with the product domain ensuring consistent variable assignments across theories.

Table 7: Capability comparison with existing tools. ✓ = supported, × = not supported, △ = partial.

Capability	TensorGuard	jaxtyping	PyTEA	TorchScript	mypy
Static (no execution)	✓	×	✓	△	✓
Zero annotations	✓	×	✓	✓	✓
Shape checking	✓	✓	✓	△	×
Device checking	✓	×	×	×	×
Phase-aware (train/eval)	✓	×	×	×	×
Symbolic dimensions	✓	✓	△	×	×
Formal soundness guarantee	✓	×	△	×	×
Counterexample generation	✓	×	✓	×	×
Parametric (all input sizes)	✓	×	×	×	×
Proof certificates	✓	×	×	×	×

12.6 False Negative Analysis

TensorGuard's recall on cross-cutting benchmarks is 0.455 (20 of 44 bugs detected). The 24 missed bugs fall into three categories:

1. **Device-only bugs without shape errors** (14 missed): When two tensors have compatible shapes but reside on different devices, the current device theory sometimes fails to detect the cross-device operation because the device constraint is only triggered when shape constraints are also present. These account for the majority of false negatives and represent the primary avenue for improving recall.

2. **Complex dynamic indexing** (6 missed): Models that use data-dependent indexing (e.g., `x[:, indices]`) produce shapes that depend on runtime values. TensorGuard conservatively marks these as Unknown, which prevents both false positives and true positive detections.

3. **Unsupported operators** (4 missed): Models using operators outside the 142-operator registry have their outputs marked Unknown, propagating inconclusive results downstream.

12.7 Verification Time Analysis

Verification times across all benchmarks follow a bimodal distribution:

- **Shallow models** (1–10 layers): 20–50 ms. Z3 solves the QF_LIA constraints almost instantly.
- **Deep models** (10–50+ layers): 50–120 ms. The dominant cost is constraint generation (AST traversal and predicate harvesting), not Z3 solving.

The IC3/PDR engine adds 30–100 ms for parametric verification, depending on the number of symbolic dimensions and the depth of the invariant search. In all experiments, IC3 converges in at most 3 frames.

12.8 Comparison with LLM-Based Bug Detection

As an additional baseline, we evaluated gpt-4.1-nano on the 56-model real-world architecture suite by providing each model's source code and asking whether it contains shape, device, or phase bugs. gpt-4.1-nano achieves:

- Precision: 0.897 (4 false positives)
- Recall: 0.972 (1 false negative)
- F1: 0.933

On the real-world suite, gpt-4.1-nano (F1 = 0.933) *outperforms* TensorGuard (F1 = 0.889). This result reflects the strength of frontier LLMs on pattern-matching tasks over natural code. However, TensorGuard provides deterministic, reproducible analysis without API dependencies, runs offline with no per-query cost,

Table 8: Phase + Shape models: per-model results (14 models, 9 TP).

Model	**Root cause**	**Det.**	**ms**
transfer_learning_phase	eval head dim mismatch	✓	42
auxiliary_head_phase	aux head wrong features	✓	38
dropout_before_fc	spatial dropout shape	×	31
batch_norm_eval	num_features eval-only	✓	45
knowledge_distill_phase	teacher dim mismatch	✓	52
label_smoothing_phase	projection dim error	✓	47
contrastive_head_phase	projection dim error	✓	39
mixup_phase	interpolation shape	×	33
stochastic_depth_phase	residual dim mismatch	✓	44
ema_update_phase	buffer size mismatch	✓	51
feature_extract_phase	output dim mismatch	✓	41
multi_task_phase	routing shape error	×	55
progressive_train_phase	feature dim error	×	48
curriculum_learn_phase	architecture shape error	×	46

Table 9: Device + Shape models: per-model results (16 models, 4 TP).

Model	**Root cause**	**Det.**	**ms**
buffer_device_mismatch	CPU buffer on GPU model	✓	35
position_ids_device	position buffer on CPU	✓	41
embedding_cpu_gpu	embedding on wrong device	×	38
layer_norm_buffer	running stats device	×	44
cross_attention_device	Q/KV device split	✓	52
multi_gpu_split	missing device transfer	×	47
data_parallel_gather	gather to wrong device	×	39
mixed_precision_cast	FP16 on CPU	×	36
buffer_no_to_device	buffer not moved	✓	43
auxiliary_output_device	aux head wrong device	×	40
skip_connection_device	skip crosses devices	×	48
batch_norm_running	running mean on CPU	×	45
gradient_checkpoint	checkpoint device issue	×	42
custom_loss_device	target on CPU	×	37
feature_pyramid_device	lateral crosses devices	×	51
attention_mask_device	mask on CPU	×	44

and catches structural composition errors (e.g., multi-hop dimension propagation across 15+ layers) that prompt-based approaches miss. LLM outputs are non-deterministic and provide no formal soundness guarantees, making them unsuitable as hard CI gates. We view the two approaches as complementary: LLMs for advisory screening, TensorGuard for formal verification.

12.9 Detailed Per-Model Results

Tables 8–10 present the complete per-model verification results for each cross-cutting category. For each model, we report the verdict (✓=detected, ×=missed), the theories exercised, the verification time, and the root cause.

Analysis of detection patterns. Several patterns emerge from the per-model results:

1. *Shape-coupled device bugs are detectable.* When the device mismatch co-occurs with a shape constraint (e.g., `buffer_device_mismatch` where the buffer has specific dimensions), TensorGuard catches the bug because the shape theory triggers constraint generation that also checks device consistency.
2. *Pure device bugs are harder.* Device-only bugs where shapes are compatible (e.g., `embedding_cpu_gpu`,

Table 10: Triple-theory models: per-model results (10 models, 6 TP).

Model	**Root cause**	**Det.**	**ms**
yolo_anchor_triple	anchor dims/CPU/eval	✓	78
mmdet_anchor_triple	channel/spatial/device	✓	82
detection_nms_triple	NMS shape/device/phase	✓	71
wav2vec2_feature_norm	buffer size/device/phase	✓	65
segmentation_mask_triple	spatial/device/phase	✓	74
object_tracking_triple	channel/device/phase	✓	69
pose_estimation_triple	heatmap spatial/dev/phase	×	88
depth_estimation_triple	scale/device/phase	×	85
optical_flow_triple	channel/device/phase	×	79
super_resolution_triple	scale factor/dev/phase	×	91

Table 11: IC3/PDR convergence on representative architectures.

Model	**Layers**	**Sym. dims**	**Frames**	**Cubes**	**ms**
ResNet-18	21	3	2	5	48
ResNet-50	54	3	2	8	67
VGG-16	16	3	2	4	41
BERT-base	14	2	2	3	39
GPT-2	14	2	2	3	42
ViT-B/16	14	3	3	6	55
U-Net	23	3	3	7	61
DCGAN generator	7	1	1	2	28
LSTM classifier	4	2	2	3	31
FPN	12	3	2	5	44

`multi_gpu_split`) are missed because the current implementation only generates device constraints when shape constraints are also present.

3. *Phase bugs with clear dimension mismatches are reliably detected.* 9 of 14 phase bugs are caught, all involving concrete dimension mismatches in the phase-conditional branch. The 5 missed bugs involve either dynamic shapes (`dropout_before_fc`), complex routing (`multi_task_phase`), or progressive architectures that change structure at training time.
4. *Triple-theory detection is strong when all three violations are shape-coupled.* 6 of 10 triple-theory bugs are detected. The 4 missed bugs involve device issues without co-occurring shape mismatches.

12.10 IC3/PDR Convergence Analysis

We present detailed IC3 convergence data for representative models.

Table 11 confirms the rapid convergence predicted by Theorem 8.10: no model requires more than 3 frames, and the number of blocking cubes is small (at most 8) even for deep architectures like ResNet-50 with 54 layers. The DCGAN generator converges in a single frame because all its layers (transposed convolutions) apply monotonically increasing spatial transformations, producing a trivially inductive invariant.

Invariant structure. The invariants discovered by IC3 have a characteristic structure: they assert that (1) all intermediate shapes are non-degenerate ($\forall i.\, \sigma_i > 0$), (2) channel dimensions match their layer parameters, and (3) spatial dimensions satisfy the divisibility constraints imposed by stride and pooling layers. For ResNet-50, the invariant is a conjunction of 8 linear constraints over the 3 symbolic dimensions (batch, height, width):

$$B \geq 1 \ \wedge\ H \geq 7 \ \wedge\ W \geq 7$$

The $H \geq 7$ and $W \geq 7$ constraints arise from the 5 stride-2 downsampling stages: the minimum valid spatial dimension after 5 halvings ($\lfloor x/2^5 \rfloor \geq 1$) requires $x \geq 32$ initially, but after accounting for padding in the initial 7×7 convolution, the effective minimum is $H, W \geq 7$.

12.11 Scalability Analysis

To characterize how verification time scales with model complexity, we measure the relationship between model size (number of layers) and verification time across all 108 benchmarks.

Constraint generation dominates. For 95% of benchmarks, Z3 solving completes in under 5 ms. The dominant cost is constraint generation: AST parsing, graph extraction, and predicate harvesting. The correlation between layer count and verification time is nearly linear:

$$t_{\text{verify}} \approx 3.2 \cdot n_{\text{layers}} + 12\,\text{ms}$$

where the 12 ms intercept accounts for Python import overhead and Z3 initialization.

Theory combination overhead. Multi-theory verification adds a constant overhead per arrangement enumerated. With typical models having at most $k_d = 4$ device variables and $k_p = 2$ phase variables, the maximum arrangement count is $S(4, 5) \cdot S(2, 2) = 15$, adding at most $15 \times 3\,\text{ms} = 45\,\text{ms}$. In practice, most arrangements are pruned early (within 1 ms) because device constraints are quickly detected as unsatisfiable.

Memory consumption. Peak memory usage is dominated by Z3's internal data structures. For the largest model in our suite (ResNet-50 with 54 layers), peak memory is 28 MB. For the smallest models (3–5 layers), peak memory is 12 MB. Memory usage is independent of input dimensions because all constraints are symbolic.

13 Implementation

TensorGuard is implemented in Python (8,408 lines in the core verifier, 306,162 lines total across the project) and requires only `z3-solver` as a runtime dependency. The tool is pip-installable and provides both a Python API and a CLI.

13.1 Z3 Integration

Verification conditions are encoded as quantifier-free formulas and submitted to Z3. TensorGuard uses Z3's `UserPropagator` API to implement custom theory plugins for $\mathcal{T}_{device}$ and $\mathcal{T}_{phase}$, which register callbacks for equality propagation and conflict detection.

UserPropagator plugins. Each finite-domain theory registers:

- `created`: register shared variables
- `fixed`: propagate when a variable is assigned a concrete value
- `eq`: propagate when two variables are equated
- `conflict`: report unsatisfiability when device/phase constraints are violated

13.2 Proof Certificates

For models verified as safe, TensorGuard extracts a proof certificate from Z3's unsatisfiability proof. The certificate encodes the inductive invariant discovered by IC3/PDR (for parametric verification) or the UNSAT core (for bounded verification), and can be independently checked.

13.3 Counterexample Generation

For models found unsafe, TensorGuard extracts a concrete counterexample from Z3's satisfying assignment. The counterexample includes:

- The failing layer and operation
- Concrete input dimensions that trigger the bug
- The expected and actual tensor metadata at the failure point
- Source location mapping to the original Python code

13.4 Testing

TensorGuard is tested by 3,691 test functions covering unit tests, integration tests, and property-based tests (via Hypothesis). Property-based tests exercise mechanized theorem statements against the Python implementation, helping close the conformance gap between the Lean specification and the implementation.

13.5 Z3 UserPropagator Details

The Z3 UserPropagator API allows registering custom theory plugins that participate in Z3's DPLL(T) search. TensorGuard registers two propagators:

Device propagator. Maintains an internal union-find structure over device variables. When Z3 assigns a device variable to a concrete value (`fixed` callback), the propagator checks all binary operation constraints involving that variable. If two operands are on different devices, it reports a conflict clause. When Z3 equates two device variables (`eq` callback), the propagator merges their equivalence classes and propagates the concrete assignment if one is known.

Phase propagator. Tracks the phase assignment (`train` or `eval`) and activates/deactivates phase-sensitive operators. When the phase variable is fixed, the propagator asserts operator-specific axioms (e.g., Dropout is inactive in eval mode).

Interaction with DPLL(T). The propagators interact with Z3's SAT solver through the following protocol:

1. Z3 makes a decision (assigns a variable).
2. Propagators check consistency and may:
    - Report a conflict (triggering backtracking).
    - Propagate new equalities/disequalities.
    - Do nothing (consistent so far).
3. Z3 continues with the next decision.

This lazy evaluation avoids eagerly enumerating all arrangements, often terminating early when a single arrangement suffices.

13.6 Source-Mapped Error Reporting

TensorGuard maps verification errors back to source code locations using AST node positions recorded during graph extraction. Each node in the computation graph retains a reference to its originating AST node (file, line, column), enabling error messages of the form:

```
model.py:15: error: shape mismatch at eval_head
  Expected in_features=128, got 256
  Note: backbone output shape is (batch, 256)
    at model.py:12: h = self.backbone(x)
  Note: eval_head declared with in_features=128
    at model.py:6: self.eval_head = nn.Linear(128, 10)
```

The error includes the chain of shape transformations leading to the mismatch, enabling rapid diagnosis.

13.7 CI/CD Integration

TensorGuard is designed for integration into continuous integration pipelines. The CLI returns exit code 0 for safe models and exit code 1 for unsafe models, supporting use as a pre-merge gate:

```
# GitHub Actions / GitLab CI
tensorguard verify models/ --recursive \
    --input-shapes config.json \
    --exit-on-first-error
```

The `--recursive` flag scans all Python files for `nn.Module` subclasses. The `--input-shapes` flag accepts a JSON configuration mapping model classes to their expected input shapes.

14 Discussion

14.1 Why Cross-Cutting Bugs Matter

The cross-cutting benchmark results (Table 3) establish a sharp discontinuity: on bugs that span multiple property domains, existing tools do not degrade gracefully—they score identically zero. This is not an artifact of benchmark design; it reflects a fundamental architectural limitation. A tool that reasons only about shapes *cannot* detect a bug that manifests only in eval mode, because it never explores the eval branch. A tool that ignores device placement *cannot* flag a cross-device operation, even if the shapes are obviously mismatched.

The 52-model benchmark is drawn from real bug patterns in production codebases (HuggingFace, torchvision, detectron2, YOLOv5, mmdetection, fairseq, timm). These are not synthetic edge cases: they represent the bugs that cause production outages when models are deployed with `model.eval()` on GPU.

14.2 The Phase Verification Gap

Phase-dependent bugs are particularly dangerous because standard testing methodology almost never catches them. Developers write tests that exercise the training loop; the model is then deployed in eval mode, where a completely different code path may be taken. TensorGuard's multi-phase verification (§4.2) addresses this gap directly, achieving $F1 = 0.783$ on the phase-dependent subset while all baselines score 0.000.

14.3 Limitations and Threats to Validity

Recall on device-only bugs. The current device theory catches device issues when they co-occur with shape mismatches, but pure cross-device operations without shape errors are sometimes missed ($\text{Recall} = 0.250$ on Device + Shape bugs). Extending the device theory to track cross-device operations through operators like `@` and `expand` is future work.

Dynamic control flow. `for` loops with data-dependent iteration counts, recursive modules, and `torch.where` with shape-dependent branches are not currently supported.

Reshape overapproximation. Complex `view()` operations with 4+ symbolic dimensions can produce false positives via overapproximation of the product-equality constraint. One false positive is observed across 230 benchmarks.

Conformance gap. The Lean mechanization proves theory combination soundness, but the Python implementation's faithfulness to the Lean specification is not machine-checked. This gap is mitigated by property-based testing but not eliminated.

Benchmark representativeness. While our benchmarks are derived from real-world bug patterns, they are hand-constructed models rather than verbatim copies of production code. The 56-model real-world suite provides external validity but does not cover the full diversity of PyTorch usage patterns.

14.4 The Theory Combination Design Space

TensorGuard's use of Tinelli–Zarba for finite-domain theories and Nelson–Oppen for stably-infinite theories reflects a deliberate design choice. Alternative approaches include:

Monolithic SMT encoding. Instead of separate theory solvers, one could encode all five theories directly in a single Z3 formula using uninterpreted functions and quantifiers. We rejected this approach because: (1) quantifiers over finite domains (device, phase) introduce nondeterminism in Z3's search; (2) the monolithic encoding is difficult to extend with new theories; and (3) the UserPropagator approach gives tighter integration with Z3's DPLL(T) loop.

Abstract interpretation product domain. The five theories could be combined as an abstract interpretation reduced product [**?**]. We chose SMT-based combination because: (1) refinement types provide a more natural specification language for tensor constraints than abstract domains; (2) SMT solving handles disjunctions (from broadcasting rules) more naturally than abstract interpretation; and (3) IC3/PDR integration is straightforward with SMT but would require significant infrastructure with abstract interpretation.

Separate analyses with post-hoc combination. One could run five independent analyses and intersect their results. This misses cross-cutting bugs by construction: a device mismatch that only manifests under specific shape constraints requires *joint* reasoning, not separate analysis.

14.5 Decidability Landscape

TensorGuard's constraint language spans several decidability classes:

- **QF_LIA** (136 operators): decidable, with Z3 producing optimal results via Simplex and branch-and-bound.
- **QF_NIA** (6 operators): undecidable in general, but TensorGuard's NIA constraints are restricted to a semi-linear subfragment (at most one symbolic factor per product term) that Z3 handles reliably via nonlinear extension of Simplex.
- **Finite-domain combination**: decidable, with complexity bounded by $S(k, n)$ (Stirling numbers) for k shared variables over an n-element domain.

The NP-hardness of reshape satisfiability (proved in the Lean mechanization) means that no algorithm can solve all reshape constraints in polynomial time (assuming $\mathrm{P} \neq \mathrm{NP}$), but the practical instances encountered in real models are well within Z3's capabilities.

14.6 Future Directions

Closing the conformance gap. The highest-impact future work is verified extraction from the Lean specification to Python (or a new verified implementation in a language with extraction support, such as OCaml or Haskell). This would eliminate the conformance gap entirely.

Extended device theory. The current device theory has limited recall on device-only bugs. Extending it to track device state through all operators (not just those with shape constraints) would improve recall on the Device + Shape category from 0.400 to an estimated 0.750+.

Dynamic control flow. Supporting `for` loops with symbolic bounds and recursive modules would extend TensorGuard's applicability to more complex architectures. This requires integrating loop invariant inference (possibly via widening) into the verification pipeline.

Gradient shape verification. TensorGuard currently verifies only forward-pass shapes. Extending to backward-pass gradient shapes would catch a new class of bugs where the gradient computation fails due to shape mismatches (e.g., custom autograd functions with incorrect output shapes).

Multi-model verification. Production ML systems often involve multiple models (teacher-student, ensemble, pipeline). Verifying shape compatibility across model boundaries is a natural extension.

Integration with type systems. Integrating TensorGuard's refinement types with Python's type system (via mypy plugins or Pyright extensions) would provide inline IDE feedback, catching shape errors as the developer types.

15 Related Work

Type-based shape checking. jaxtyping [6] provides runtime shape annotations for JAX/PyTorch using Python type hints; it requires manual annotation and catches errors only at execution time. tsanley [18] infers named dimensions from runtime traces but does not verify them statically. Neither tool reasons about device placement, training phase, or cross-cutting property interactions. Einops provides a DSL for shape transformations with implicit shape checks, but does not verify entire architectures.

Static shape analysis. PyTEA [12] performs static type analysis for PyTorch tensor shapes via abstract interpretation. It targets individual operations and does not reason about device placement, training phase, or architecture-level composition via SMT theory combination. TensorFlow's shape inference operates at graph construction time but is limited to the TensorFlow computation model. ShapeFlow infers shapes for TensorFlow programs but does not handle PyTorch's imperative model definition style. No existing static shape analyzer verifies the cross-cutting property interactions that TensorGuard targets.

SMT-based program verification. Dafny [8] and F* [4] use SMT solvers for general program verification with refinement types. Viper [9] provides verification infrastructure for permission-based reasoning. CBMC performs bounded model checking for C programs. TensorGuard applies similar SMT-based techniques but specializes them with domain-specific tensor shape theories and UserPropagator plugins for finite-domain sorts. The key difference is that TensorGuard combines five domain-specific theories via Tinelli–Zarba, rather than relying on a single general-purpose SMT theory.

Refinement types. Liquid Types [3] pioneered practical refinement type checking via SMT, inferring refinement types automatically using predicate abstraction. RefinedC [13] extends refinement types to C with ownership semantics. Stardust adds refinement types to ML with liquid type inference. Flux extends Rust with refinement types for safe systems programming. TensorGuard adapts the refinement type paradigm to tensor shape verification with domain-specific theories, contributing the first application of Tinelli–Zarba theory combination to a refinement type system. Unlike Liquid Haskell, which operates in a single theory (QF_LIA for integer refinements), TensorGuard combines five theories with both stably-infinite and finite-domain sorts.

Abstract interpretation for neural networks. Singh et al. [15] develop abstract domains for certifying neural network properties (robustness, fairness). ERAN provides certified defense against adversarial examples. α-β-CROWN uses branch-and-bound with linear relaxations. These tools target network *behavior* (output properties) rather than *structural* shape safety (dimension compatibility), and operate on trained model weights rather than source code. TensorGuard is complementary: it verifies that the network is structurally well-formed, while these tools verify that a well-formed network behaves correctly.

Model checking for software. IC3/PDR [1, 2] has been applied to hardware verification and software model checking (e.g., Spacer in Z3). TensorGuard's contribution is the observation that tensor shape verification can be cast as a safety property over a Kripke structure, enabling parametric verification over all input dimensions. The rapid convergence of IC3 frames (at most 3 for all evaluated models) suggests that neural network computation graphs have particularly favorable structure for property-directed reachability.

Deep learning bug studies. Islam et al. [5] and Zhang et al. [19] empirically characterize DL bugs, identifying shape mismatches as the largest category. CRADLE detects library bugs via differential testing. DeepLocalize localizes faults using dynamic analysis. TensorGuard is the first static tool to address the cross-cutting property interactions these studies identify as problematic.

16 Conclusion

We have presented TensorGuard, a static multi-property verifier for PyTorch that combines five domain-specific SMT theories—shape, device, phase, stride, and permutation—via Tinelli–Zarba and Nelson–Oppen theory combination to catch cross-cutting bugs that no existing tool detects.

The headline result is stark: on a 52-model cross-cutting benchmark sourced from real bugs in HuggingFace, torchvision, detectron2, YOLOv5, and mmdetection, TensorGuard achieves $\mathrm{F1} = 0.625$ with perfect precision while no baseline tool is designed to detect these cross-cutting bug classes. On a 56-model real-world suite, F1 reaches 0.889 (precision 0.889, 4 false positives). While frontier LLMs achieve competitive F1 (gpt-4.1-nano: $\mathrm{F1} = 0.933$ on the real-world suite), TensorGuard provides deterministic, reproducible analysis without API dependencies. Verification completes in under 120 ms.

TensorGuard is not an incremental improvement over existing tools. It creates a new category of multi-property deep learning verification, detecting bugs that exist at the intersection of shape, device, and phase—exactly the bugs that cause production failures when models are deployed.

Future work includes closing the conformance gap between the mechanized Lean specification and the Python implementation via verified extraction, extending device theory coverage, and integrating TensorGuard into CI/CD pipelines as a pre-merge gate.

References

[1] A. R. Bradley. SAT-based model checking without unrolling. In *VMCAI*, pages 70–87. Springer, 2011.

[2] N. Eén, A. Mishchenko, and R. Brayton. Efficient implementation of property directed reachability. In *FMCAD*, pages 125–134, 2011.

[3] P. M. Rondon, M. Kawaguchi, and R. Jhala. Liquid types. In *PLDI*, pages 159–169. ACM, 2008.

[4] N. Swamy, C. Hriţcu, C. Keller, A. Rastogi, A. Delignat-Lavaud, S. Forest, K. Bhargavan, C. Fournet, P.-Y. Strub, M. Kohlweiss, J.-K. Zinzindohoué, and S. Zanella-Béguelin. Dependent types and multi-monadic effects in F*. In *POPL*, pages 256–270. ACM, 2016.

[5] M. J. Islam, G. Nguyen, R. Pan, and H. Rajan. A comprehensive study on deep learning bug characteristics. In *ESEC/FSE*, pages 510–520. ACM, 2019.

[6] P. Kidger. jaxtyping: Type annotations for JAX. `https://github.com/google/jaxtyping`, 2023.

[7] J.-L. Lassez, M. Maher, and K. Marriott. Unification revisited. In *Foundations of Deductive Databases and Logic Programming*, pages 587–625. Morgan Kaufmann, 1988.

[8] K. R. M. Leino. Dafny: An automatic program verifier for functional correctness. In *LPAR*, pages 348–370. Springer, 2010.

[9] P. Müller, M. Schwerhoff, and A. J. Summers. Viper: A verification infrastructure for permission-based reasoning. In *VMCAI*, pages 41–62. Springer, 2016.

[10] G. Nelson and D. C. Oppen. Simplification by cooperating decision procedures. *ACM TOPLAS*, 1(2):245–257, 1979.

[11] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala. PyTorch: An imperative style, high-performance deep learning library. In *NeurIPS*, pages 8024–8035, 2019.

[12] H. Y. Jhoo, S. Kim, W. Song, K. Park, D. Lee, and K. Yi. A Static Analyzer for Detecting Tensor Shape Errors in Deep Neural Network Training Code. In *ECOOP*, 2023.

[13] M. Sammler, R. Lepigre, R. Krebbers, K. Memarian, D. Dreyer, and D. Garg. RefinedC: Automating the foundational verification of C code with refined ownership types. In *PLDI*, pages 158–174. ACM, 2021.

[14] H. G. Rice. Classes of recursively enumerable sets and their decision problems. *Trans. AMS*, 74(2):358–366, 1953.

[15] G. Singh, T. Gehr, M. Püschel, and M. Vechev. An abstract domain for certifying neural networks. *Proc. ACM Program. Lang.*, 3(POPL):41:1–41:30, 2019.

[16] C. Tinelli and C. G. Zarba. Combining nonstably infinite theories. *J. Automated Reasoning*, 34(3):209–238, 2005.

[17] PyTorch Contributors. TorchScript. `https://pytorch.org/docs/stable/jit.html`, 2023.

[18] A. Rogozhnikov. tsanley: Understanding tensor shapes in Python. `https://github.com/arogozhnikov/tsanley`, 2022.

[19] Y. Zhang, Y. Chen, S.-C. Cheung, Y. Xiong, and L. Zhang. An empirical study on TensorFlow program bugs. In *ISSTA*, pages 129–140. ACM, 2018.

A Complete Operator List

The 142 operators supported by TensorGuard are organized into the following categories. Each entry shows the operator name and the SMT fragment of its generated constraints.

Convolution (12): Conv1d, Conv2d, Conv3d, ConvTranspose1d, ConvTranspose2d, ConvTranspose3d, and their lazy variants (LazyConv1d, LazyConv2d, LazyConv3d, LazyConvTranspose1d, LazyConvTranspose2d, LazyConvTranspose3d). All QF_LIA.

Normalization (15): BatchNorm1d, BatchNorm2d, BatchNorm3d, LayerNorm, GroupNorm, InstanceNorm1d, InstanceNorm2d, InstanceNorm3d, SyncBatchNorm, LazyBatchNorm1d, LazyBatchNorm2d, LazyBatchNorm3d, LazyInstanceNorm1d, LazyInstanceNorm2d, LazyInstanceNorm3d. All QF_LIA.

Attention (4): MultiheadAttention, TransformerEncoder, TransformerDecoder, TransformerEncoderLayer. QF_LIA.

Recurrence (6): LSTM, GRU, RNN, LSTMCell, GRUCell, RNNCell. QF_LIA.

Pooling (16): MaxPool1d, MaxPool2d, MaxPool3d, AvgPool1d, AvgPool2d, AvgPool3d, AdaptiveMaxPool1d, AdaptiveMaxPool2d, AdaptiveMaxPool3d, AdaptiveAvgPool1d, AdaptiveAvgPool2d, AdaptiveAvgPool3d, LPPool1d, LPPool2d, FractionalMaxPool2d, FractionalMaxPool3d. All QF_LIA.

Activation (25): ReLU, LeakyReLU, PReLU, ELU, SELU, GELU, CELU, Sigmoid, Tanh, Softmax, LogSoftmax, Hardswish, Hardsigmoid, Hardtanh, Hardshrink, Mish, SiLU, GLU, Softplus, Softshrink, Softsign, Tanhshrink, Threshold, RReLU, SELU. Shape-preserving (trivial constraints).

Padding (15): ZeroPad1d, ZeroPad2d, ZeroPad3d, ConstantPad1d, ConstantPad2d, ConstantPad3d, ReflectionPad1d, ReflectionPad2d, ReflectionPad3d, ReplicationPad1d, ReplicationPad2d, ReplicationPad3d, CircularPad1d, CircularPad2d, CircularPad3d. All QF_LIA.

Loss (21): CrossEntropyLoss, MSELoss, NLLLoss, BCELoss, BCEWithLogitsLoss, L1Loss, SmoothL1Loss, KLDivLoss, HuberLoss, CTCLoss, CosineEmbeddingLoss, HingeEmbeddingLoss, MarginRankingLoss, MultiMarginLoss, MultiLabelMarginLoss, MultiLabelSoftMarginLoss, SoftMarginLoss, TripletMarginLoss, TripletMarginWithDistanceLoss, PoissonNLLLoss, GaussianNLLLoss. All QF_LIA.

Embedding (2): Embedding, EmbeddingBag. QF_LIA.

Structural (26): reshape, view, flatten, unflatten, cat, stack, split, chunk, squeeze, unsqueeze, transpose, permute, contiguous, expand, repeat, narrow, select, index_select, gather, scatter, PixelShuffle, PixelUnshuffle, Fold, Unfold, ChannelShuffle, Upsample. reshape/view/flatten/unflatten/PixelShuffle/repeat: QF_NIA; others: QF_LIA.

B Trusted Computing Base

The trusted computing base (TCB) of TensorGuard comprises:

1. **Z3 SMT solver.** We trust Z3's decision procedures for QF_LIA, QF_NIA, and QF_UF.
2. **Lean 4 draft proofs.** A Lean proof checker would enter the TCB only after the current formalization draft is packaged and compiled. At present, no `lakefile.lean` is present, so the theorem statements and proof terms should be read as an uncompiled supplementary artifact rather than a completed machine-checked proof.
3. **Python implementation.** The TensorGuard Python code (graph extraction, transfer functions, constraint generation) is *not* mechanically verified. The conformance gap is mitigated by property-based testing (Hypothesis) and 3,691 test functions.
4. **Transfer function faithfulness.** Each of the 142 transfer functions is assumed to faithfully model the corresponding PyTorch operator, validated by property-based testing against the PyTorch runtime.
5. **Graph extraction correctness.** The AST-based and FX-based extractors are assumed to correctly reconstruct the computation graph. Dynamic dispatch, monkey-patching, and metaclass usage can cause extraction failures (reported as warnings, not silent misses).

C Proof Sketches

C.1 Stride Theory Convexity

Proof. $\mathcal{T}_{stride}$ generates constraints in QF_LIA over $\mathbb{Z}_{\geq 1}$. QF_LIA is convex: if $\varphi \models e_1 = e_2 \vee e_3 = e_4$, then $\varphi \models e_1 = e_2$ or $\varphi \models e_3 = e_4$. This follows from the fact that the solution set of a conjunction of linear inequalities over the integers is a finite union of lattice points in a convex polytope, and equality disjunctions over such sets reduce to individual equalities [7].

The NIA subfragment arising from reshape constraints uses at most one symbolic factor per product term, restricting to a semi-linear fragment where convexity is preserved. □

C.2 NP-Hardness of Reshape Satisfiability

Theorem C.1 (Reshape NP-Hardness)**.** *The reshape feasibility problem—given symbolic shape tuples s and d, is $\prod_i s_i = \prod_j d_j$ satisfiable?—is NP-hard.*

Proof sketch. By reduction from SUBSET-PRODUCT. Given a SUBSET-PRODUCT instance $(a_1, \ldots, a_n, T)$, construct a reshape constraint where the input shape dimensions are $a_1, \ldots, a_n$ (each optionally included via a binary selector variable $b_i \in \{1, a_i\}$) and the target product is T. The reshape constraint $\prod_i (b_i \cdot a_i + (1 - b_i)) = T$ is satisfiable iff the SUBSET-PRODUCT instance has a solution. This reduction is mechanized in the Lean 4 formalization. □

D Theory Combination Correctness

We present extended proofs and discussions of theory combination correctness.

D.1 Tinelli–Zarba Soundness Proof

Theorem D.1 (Tinelli–Zarba Soundness, restated)**.** *Let $\mathcal{T}_1, \ldots, \mathcal{T}_n$ be theories with pairwise disjoint signatures. Let some theories be stably-infinite and others have finite-domain sorts. The arrangement-based combination procedure decides satisfiability of $\varphi_1 \wedge \cdots \wedge \varphi_n$ correctly.*

Proof. We prove both directions.

Soundness ($\Rightarrow$): If the procedure returns SAT with arrangement α, then:

1. Each $\varphi_i \wedge \mathit{constr}(\alpha)$ is satisfiable in $\mathcal{T}_i$ (by Z3's soundness).
2. The arrangement α specifies equalities and disequalities over shared variables consistently with each finite domain's cardinality.
3. By the Nelson–Oppen lemma (for stably-infinite theories under arrangement α), the satisfying models can be combined into a single model satisfying all φ_i simultaneously.

Completeness ($\Leftarrow$): If $\varphi_1 \wedge \cdots \wedge \varphi_n$ is satisfiable, then there exists a combined model M. The restriction of M to each finite-domain sort induces an equivalence relation on shared variables—an arrangement α^*. Since M satisfies each φ_i, each $\varphi_i \wedge \mathit{constr}(\alpha^*)$ is satisfiable. The procedure enumerates all arrangements including α^*, so it will find a satisfying arrangement. ☐

D.2 Convexity of QF_LIA

Lemma D.2 (QF_LIA Convexity)**.** *If φ is a satisfiable QF_LIA formula and $\varphi \models (s = t) \vee (u = v)$, then $\varphi \models (s = t)$ or $\varphi \models (u = v)$.*

Proof. Suppose $\varphi \models (s = t) \vee (u = v)$ but $\varphi \not\models (s = t)$ and $\varphi \not\models (u = v)$. Then there exist models $M_1 \models \varphi$ with $M_1(s) \neq M_1(t)$ and $M_2 \models \varphi$ with $M_2(u) \neq M_2(v)$. Since φ is a conjunction of linear inequalities, its solution set is a convex polytope (intersected with $\mathbb{Z}^n$).

Since $M_1 \models \varphi$ and $M_1(s) \neq M_1(t)$, we have $M_1(s = t)$ is false, so $M_1(u = v)$ must be true (by the disjunction). Similarly, $M_2(s = t)$ must be true. But M_1 and M_2 both satisfy φ, and convexity of the solution set ensures that any convex combination also satisfies φ.

Consider the midpoint $M_3 = \frac{1}{2}(M_1 + M_2)$ (extended to the integer lattice). Since $M_1(u) = M_1(v)$ and $M_2(s) = M_2(t)$, but $M_1(s) \neq M_1(t)$ and $M_2(u) \neq M_2(v)$, the midpoint may violate both equalities, contradicting $\varphi \models (s = t) \vee (u = v)$.

The formal argument uses the Lassez–Maher–Marriott characterization [7] of integer linear arithmetic solution sets. ☐

D.3 Arrangement Complexity Analysis

Proposition D.3 (Arrangement Count)**.** *For k shared variables over a finite domain of cardinality n, the number of arrangements is the Stirling number of the second kind $S(k, \min(k, n))$, which satisfies:*

$$S(k, n) = \frac{1}{n!} \sum_{j=0}^{n} (-1)^j \binom{n}{j} (n - j)^k$$

Practical bounds. In TensorGuard's product domain:

- $\mathcal{T}_{device}$: $|\mathcal{D}| = 5$, typically $k_d \leq 4$ shared device variables. $S(4, 4) = 15$ arrangements.
- $\mathcal{T}_{phase}$: $|\mathcal{P}| = 2$, typically $k_p \leq 2$ shared phase variables. $S(2, 2) = 1$ arrangement.
- Total: at most $15 \cdot 1 = 15$ arrangement combinations in typical models. Even in pathological cases with $k_d = 8$, $S(8, 5) = 11{,}501$—still tractable.

E Complete Benchmark Listings

E.1 Cross-Cutting Benchmark Categories

We list all 52 models in the cross-cutting benchmark, organized by category.

Device + Shape (16 models).

1. `buffer_device_mismatch`: `register_buffer` scale tensor on CPU, model on GPU. $\mathcal{T}_{device} \times \mathcal{T}_{shape}$.
2. `position_ids_device`: Position ID buffer stays on CPU. Modeled after HuggingFace #13666.
3. `embedding_cpu_gpu`: Embedding table on CPU, input on GPU.
4. `layer_norm_buffer_device`: LayerNorm running statistics buffer on wrong device.
5. `cross_attention_device`: Cross-attention with Q on GPU, K/V on CPU.
6. `multi_gpu_split`: Model split across GPUs with missing transfer at boundary.
7. `data_parallel_gather`: DataParallel gather to wrong device.
8. `mixed_precision_cast`: FP16 cast on CPU with FP32 computation on GPU.
9. `buffer_no_to_device`: Buffer created with `torch.zeros` (CPU) not moved with model.
10. `auxiliary_output_device`: Auxiliary output head on different device.
11. `skip_connection_device`: Skip connection crosses device boundary.
12. `batch_norm_running_device`: BatchNorm running mean on CPU.
13. `gradient_checkpoint_device`: Gradient checkpointing with device mismatch.
14. `custom_loss_device`: Custom loss function with target on CPU.
15. `feature_pyramid_device`: FPN lateral connection crosses devices.
16. `attention_mask_device`: Attention mask created on CPU, attention on GPU.

Phase + Shape (14 models).

1. `transfer_learning_phase`: Eval head with wrong dimensions (Listing 1).
2. `auxiliary_head_phase`: Auxiliary head only used in training, wrong dims.
3. `dropout_before_fc`: Dropout changes effective shape in eval (spatial dropout).
4. `batch_norm_eval`: BatchNorm with wrong num_features, only crashes in eval due to running stats.
5. `knowledge_distillation_phase`: Teacher branch with dimension mismatch, only active in training.
6. `label_smoothing_phase`: Label smoothing projection with wrong dims, training-only. Modeled after fairseq.
7. `contrastive_head_phase`: Contrastive learning head with wrong projection dims, training-only.
8. `mixup_phase`: Mixup augmentation with shape error, training-only.
9. `stochastic_depth_phase`: Stochastic depth with residual mismatch, training-only.
10. `ema_update_phase`: EMA model update with wrong buffer sizes, training-only.
11. `feature_extraction_phase`: Feature extraction mode with wrong output dims.
12. `multi_task_phase`: Multi-task heads with phase-dependent routing.
13. `progressive_training_phase`: Progressive resizing with wrong feature dims.
14. `curriculum_learning_phase`: Curriculum schedule changes effective architecture.

Device + Phase (4 models).

1. `eval_on_gpu_buffer_cpu`: Model `.eval().cuda()` with CPU buffer.
2. `training_cpu_inference_gpu`: Model trained on CPU, deployed on GPU without moving all buffers.
3. `phase_dependent_device_transfer`: Device transfer only in eval branch.
4. `distributed_eval_device`: Distributed training to single-GPU eval with device mismatch.

Triple-theory (10 models).

1. `yolo_anchor_triple`: Anchor grid with wrong dims, CPU, eval-only (Listing 3).
2. `mmdet_anchor_triple`: Anchor generator with channel/spatial/device mismatch, eval-only.
3. `detection_nms_triple`: NMS post-processing with shape/device/phase errors.
4. `wav2vec2_feature_norm`: Feature normalization buffer with size/device/phase mismatch.
5. `segmentation_mask_triple`: Segmentation mask with spatial/device/phase errors.
6. `object_tracking_triple`: Tracking head with channel/device/phase errors.
7. `pose_estimation_triple`: Pose heatmap with spatial/device/phase errors.
8. `depth_estimation_triple`: Depth decoder with scale/device/phase errors.
9. `optical_flow_triple`: Flow estimation with channel/device/phase errors.
10. `super_resolution_triple`: Upsampling with scale factor/device/phase errors.

Correct models (8 models).

1. `correct_resnet_phase`: ResNet with correct dropout in both phases.
2. `correct_bert_device`: BERT with correctly transferred buffers.
3. `correct_yolo_triple`: YOLO with correct anchors, device, and phase.
4. `correct_unet_skip`: U-Net with correct skip connections.
5. `correct_gan_phase`: GAN with correct phase handling.
6. `correct_transformer_device`: Transformer with correct device management.
7. `correct_multitask_phase`: Multi-task model with correct phase routing.
8. `correct_fpn_triple`: FPN with correct channels, device, and phase.

E.2 Real-World Architecture Suite

The 56-model real-world architecture suite includes models from the following families:

- **Convolutional**: ResNet (FC bug, bottleneck bug, downsample bug), VGG (classifier mismatch), MobileNet (inverted residual bug), EfficientNet (compound scaling bug), DenseNet (growth rate bug), U-Net (skip connection bug), DCGAN (generator bug)
- **Transformer**: BERT (pooler bug, attention bug), GPT (projection bug), ViT (patch embedding bug, cls token bug), DeiT (distillation head bug)
- **Hybrid**: FPN (lateral connection bug), Detection heads (RPN bug, ROI align bug), Segmentation (decoder bug)
- **Recurrent**: LSTM classifier (hidden size bug), GRU sequence-to-sequence (encoder-decoder bug)
- **Correct models** (20): One correct variant for each buggy family, plus additional architecturally diverse models

F Lean Mechanization Details

The repository contains a 1,587-line Lean 4 formalization draft in `TheoryCombination.lean` with 59 stated theorems and zero `sorry` obligations. **However,** no `lakefile.lean` or `lean-toolchain` file is present, so the draft has not been compiled with `lake build` and should not be described as a completed machine-checked development. The theorem statements and proof structure have been written and reviewed; we summarize the key theorems.

F.1 Key Theorems

1. `combination_soundness`: The Tinelli–Zarba procedure is sound—if it returns SAT with arrangement α, then the combined formula is satisfiable.

2. `tensorguard_combination_sound`: The specific instantiation for TensorGuard's five-theory product domain is sound.

3. `arrangement_count_bound`: The number of arrangements is bounded by the Stirling number $S(k, n)$.

4. `broadcast_sound`: Broadcasting preserves element-count compatibility.

5. `stride_sound`: Stride constraints are consistent with shape constraints.

6. `device_consistent_transitive`: Device consistency is a transitive relation.

7. `broadcast_symmetric`: Broadcasting is symmetric ($\text{broadcast}(a, b) = \text{broadcast}(b, a)$).

8. `matmul_sound`: Matrix multiplication shape constraints are sound.

9. `matmul_chain_dims`: Chained matmul dimensions are consistent.

10. `linear_output_dim`: Linear layer output dimension is correct.

11. `mha_head_dim_sound`: Multi-head attention head dimension constraint is sound.

12. `broadcastDim_sound`, `broadcastDim_complete`: Broadcasting dimension computation is both sound and complete.
13. `strideDim_sound`, `strideDim_complete`: Stride dimension computation is both sound and complete.
14. `deviceCheck_sound`, `deviceCheck_complete`: Device compatibility checking is both sound and complete.
15. `phaseCheck_sound`, `phaseCheck_complete`: Phase compatibility checking is both sound and complete.

F.2 Proof Structure

The mechanization is organized into three layers:

Layer 1: Theory signatures and models. Defines the abstract interface for theories (sorts, function symbols, predicate symbols) and models (interpretations of signatures). This layer is independent of tensor verification.

Layer 2: Combination procedure. Formalizes the Tinelli–Zarba procedure, arrangements, and the Nelson–Oppen equality propagation loop. Proves the main soundness and completeness theorems.

Layer 3: Tensor-specific instantiation. Instantiates the abstract combination framework with TensorGuard's five theories. Proves that each theory satisfies the interface contracts (signature disjointness, stable infiniteness or finite-domain declaration, convexity for stably-infinite theories).

G Z3 Encoding Details

We present the Z3 encoding for representative operators, showing the actual SMT-LIB formulas generated by TensorGuard.

G.1 Linear Layer Encoding

For `nn.Linear(128, 64)` applied to input variable `x`:

```
; Input shape variables
(declare-const x_ndim Int)
(declare-const x_dim_last Int)
; Output shape variables
(declare-const y_ndim Int)
(declare-const y_dim_last Int)
; Precondition: last dimension = in_features
(assert (= x_dim_last 128))
; Transfer function: output preserves all dims
; except last, which becomes out_features
(assert (= y_ndim x_ndim))
(assert (= y_dim_last 64))
; Positive dimension constraints
(assert (>= x_dim_last 1))
(assert (>= y_dim_last 1))
```

Listing 8: Z3 encoding of Linear(128, 64) shape constraints.

G.2 Conv2d Encoding

For `nn.Conv2d(3, 64, kernel_size=7, stride=2, padding=3)`:

```
; Input: (batch, 3, H, W)
(declare-const x_dim0 Int)  ; batch
(declare-const x_dim1 Int)  ; channels
(declare-const x_dim2 Int)  ; height
(declare-const x_dim3 Int)  ; width
; Output: (batch, 64, H', W')
(declare-const y_dim0 Int)
(declare-const y_dim1 Int)
(declare-const y_dim2 Int)
(declare-const y_dim3 Int)
; Precondition: 4D input, correct in_channels
(assert (= x_dim1 3))
; Transfer: batch preserved, channels = out_channels
(assert (= y_dim0 x_dim0))
(assert (= y_dim1 64))
; Spatial: floor((H + 2*3 - 1*(7-1) - 1) / 2) + 1
(assert (= y_dim2 (+ (div (- (+ x_dim2 6) 7) 2) 1)))
(assert (= y_dim3 (+ (div (- (+ x_dim3 6) 7) 2) 1)))
```

Listing 9: Z3 encoding of Conv2d shape constraints.

G.3 Cross-Theory Device + Shape Encoding

For a model with a registered buffer on CPU and computation on GPU:

```
; Device variables (0=cpu, 1=cuda:0, ...)
(declare-const x_device Int)
(declare-const buf_device Int)
(declare-const y_device Int)
; Shape variables
(declare-const x_dim1 Int)
(declare-const buf_dim0 Int)
; Input on GPU
(assert (= x_device 1))
; Buffer registered on CPU
(assert (= buf_device 0))
; Binary operation requires same device
(assert (= x_device buf_device))  ; CONFLICT
; Shape compatibility
(assert (= x_dim1 buf_dim0))
```

Listing 10: Cross-theory encoding for device + shape constraints.

Z3 immediately detects the conflict: $x_{\text{device}} = 1$ and $\text{buf}_{\text{device}} = 0$ contradict the same-device precondition. The UserPropagator for $\mathcal{T}_{device}$ reports this as a conflict clause before shape checking even begins.

G.4 Phase-Conditional Encoding

For a model with `if self.training:` branching:

```
; Phase variable (0=train, 1=eval)
(declare-const mode Int)
; Train branch shape variables
(declare-const train_out_dim Int)
; Eval branch shape variables
```

```
(declare-const eval_out_dim Int)
; Backbone output
(declare-const backbone_dim Int)
(assert (= backbone_dim 256))
; Train head: Linear(256, 100) - OK
(assert (=> (= mode 0)
    (= train_out_dim 100)))
; Eval head: Linear(128, 10) - BUG
(assert (=> (= mode 1)
    (and (= backbone_dim 128)  ; precondition
         (= eval_out_dim 10))))
; Check both phases
(push)
(assert (= mode 0))
(check-sat)  ; SAT - train branch OK
(pop)
(push)
(assert (= mode 1))
(check-sat)  ; UNSAT - 256 != 128
(pop)
```

Listing 11: Phase-conditional verification encoding.

H Additional Transfer Function Details

We present detailed transfer functions for operators beyond the core rules in §6, including loss functions, recurrence operators, and structural operations.

H.1 Loss Function Transfer Functions

Loss functions typically reduce their inputs, producing scalar or reduced-dimension outputs.

T-CrossEntropyLoss.

$$\frac{\Gamma \vdash x : \{t \mid \mathit{ndim}(t) = 2 \wedge \mathit{dim}_0(t) = N\} \Gamma \vdash y : \{t \mid \mathit{shape}(t) = (N,)\}}{\Gamma \vdash \texttt{CrossEntropyLoss}()(x, y) : \{t' \mid \mathit{shape}(t') = ()\}} \text{ T-XEnt}$$

When `reduction='none'`:

$$\frac{\Gamma \vdash x : \{t \mid \mathit{ndim}(t) = 2 \wedge \mathit{dim}_0(t) = N\} \Gamma \vdash y : \{t \mid \mathit{shape}(t) = (N,)\}}{\Gamma \vdash \texttt{CrossEntropyLoss}(\text{none})(x, y) : \{t' \mid \mathit{shape}(t') = (N,)\}} \text{ T-XEnt-none}$$

T-MSELoss.

$$\frac{\Gamma \vdash x : \{t \mid \mathit{shape}(t) = s\} \Gamma \vdash y : \{t \mid \mathit{shape}(t) = s\}}{\Gamma \vdash \texttt{MSELoss}()(x, y) : \{t' \mid \mathit{shape}(t') = ()\}} \text{ T-MSE}$$

H.2 Recurrence Operator Details

T-GRU.

$$\frac{\Gamma \vdash x : \{t \mid \mathit{ndim}(t) = 3 \wedge \mathit{dim}_2(t) = f_{\text{in}}\}}{\Gamma \vdash \texttt{GRU}(f_{\text{in}}, h, L, bi)(x) : \{(o, h_n) \mid \mathit{dim}_2(o) = h \cdot (1 + bi) \wedge \mathit{dim}_0(h_n) = L \cdot (1 + bi)\}} \text{ T-GRU}$$

H.3 Embedding Operator Details

T-Embedding.

$$\frac{\Gamma \vdash x : \{t \mid \mathit{dtype}(t) \in \{\texttt{int32}, \texttt{int64}\}\} \quad V, D \in \mathbb{Z}_{>0}}{\Gamma \vdash \texttt{Embedding}(V, D)(x) : \{t' \mid \mathit{shape}(t') = \mathit{shape}(x) \parallel [D]\}} \text{ T-Embed}$$

T-EmbeddingBag.

$$\frac{\Gamma \vdash x : \{t \mid \mathit{ndim}(t) = 1\} \qquad V, D \in \mathbb{Z}_{>0}}{\Gamma \vdash \texttt{EmbeddingBag}(V, D)(x) : \{t' \mid \mathit{shape}(t') = (B, D)\}} \text{ T-EmbedBag}$$

where B is the number of bags (determined by the offsets argument).

H.4 Padding Operator Details

T-ZeroPad2d.

$$\frac{\Gamma \vdash x : \{t \mid \mathit{ndim}(t) \geq 2\} \qquad p = (p_l, p_r, p_t, p_b)}{\Gamma \vdash \texttt{ZeroPad2d}(p)(x) : \{t' \mid \mathit{dim}_{-1}(t') = \mathit{dim}_{-1}(x) + p_l + p_r \wedge \mathit{dim}_{-2}(t') = \mathit{dim}_{-2}(x) + p_t + p_b\}} \text{ T-ZeroPad2d}$$

T-ConstantPad3d.

$$\frac{\Gamma \vdash x : \{t \mid \mathit{ndim}(t) \geq 3\} \qquad p = (p_1, \ldots, p_6)}{\Gamma \vdash \texttt{ConstantPad3d}(p, c)(x) : \{t' \mid \forall i \in \{1, 2, 3\}.\ \mathit{dim}_{-i}(t') = \mathit{dim}_{-i}(x) + p_{2i-1} + p_{2i}\}} \text{ T-ConstPad3d}$$

H.5 Structural Operator Details

T-Squeeze.

$$\frac{\Gamma \vdash x : \{t \mid \mathit{shape}(t) = s \wedge s_d = 1\}}{\Gamma \vdash \texttt{squeeze}(x, d) : \{t' \mid \mathit{shape}(t') = s_{[0:d]} \parallel s_{[d+1:]}\}} \text{ T-Squeeze}$$

T-Unsqueeze.

$$\frac{\Gamma \vdash x : \{t \mid \mathit{shape}(t) = s\} \qquad 0 \leq d \leq \mathit{ndim}(x)}{\Gamma \vdash \texttt{unsqueeze}(x, d) : \{t' \mid \mathit{shape}(t') = s_{[0:d]} \parallel [1] \parallel s_{[d:]}\}} \text{ T-Unsqueeze}$$

T-Expand.

$$\frac{\Gamma \vdash x : \{t \mid \mathit{shape}(t) = s\} \qquad \forall i.\ d_i = s_i \vee s_i = 1}{\Gamma \vdash \texttt{expand}(x, d_1, \ldots, d_k) : \{t' \mid \mathit{shape}(t') = (d_1, \ldots, d_k)\}} \text{ T-Expand}$$

T-Repeat.

$$\frac{\Gamma \vdash x : \{t \mid \mathit{shape}(t) = s\} \qquad |r| \geq |s|}{\Gamma \vdash \texttt{repeat}(x, r_1, \ldots, r_k) : \{t' \mid \forall i.\ \mathit{dim}_i(t') = s_i \cdot r_i\}} \text{ T-Repeat}$$

T-Stack.

$$\frac{\Gamma \vdash t_i : \{t \mid \mathit{shape}(t) = s\} \quad (i = 1, \ldots, n)\ 0 \leq d \leq \mathit{ndim}(t_1)}{\Gamma \vdash \texttt{stack}([t_1, \ldots, t_n], d) : \{t' \mid \mathit{shape}(t') = s_{[0:d]} \parallel [n] \parallel s_{[d:]}\}} \text{ T-Stack}$$

T-Chunk.

$$\frac{\Gamma \vdash x : \{t \mid \mathit{shape}(t) = s\} \qquad s_d \bmod n = 0}{\Gamma \vdash \texttt{chunk}(x, n, d) : [t'_1, \ldots, t'_n] \text{ where } \forall i.\ \mathit{dim}_d(t'_i) = s_d / n} \text{ T-Chunk}$$

T-PixelShuffle.

$$\frac{\Gamma \vdash x : \{t \mid \mathit{ndim}(t) = 4 \wedge \mathit{dim}_1(t) = C \wedge C \bmod r^2 = 0\}}{\Gamma \vdash \texttt{PixelShuffle}(r)(x) : \{t' \mid \mathit{dim}_1(t') = C/r^2 \wedge \mathit{dim}_2(t') = \mathit{dim}_2(x) \cdot r \wedge \mathit{dim}_3(t') = \mathit{dim}_3(x) \cdot r\}} \text{ T-PixelShuffle}$$

This is one of the QF_NIA operators: the constraint $C \bmod r^2 = 0$ involves nonlinear arithmetic.

I Experimental Reproducibility

I.1 Environment

All experiments were conducted on:

- Hardware: Apple M-series processor, 16 GB RAM
- Python 3.11
- Z3 4.12.2
- Lean 4.8.0 (for mechanization; note: proofs have not yet been compiled—no `lakefile.lean` is present)

I.2 Reproducing Results

```
# Install dependencies
pip install numpy z3-solver

# Cross-cutting multi-theory evaluation (Table 2)
python3 experiments/run_multi_theory_eval.py

# Real-world architecture evaluation (Table 4)
python3 experiments/run_realworld_pytorch_eval.py

# Shape-only bug detection (Table 5)
python3 experiments/run_real_baseline_comparison.py

# IC3/PDR parametric verification
python3 src/ic3_pdr.py

# Run all tests
python3 -m pytest tests/ -x -q

# Verify Lean mechanization (requires lakefile.lean setup)
# Note: lakefile.lean is not yet present in the repository
cd lean && lake build
```

Listing 12: Commands to reproduce all experimental results.